

Hochschule für Angewandte Wissenschaften Hamburg

Fakultät Technik und Informatik

Department Informatik

Zero-Knowledge-Netzwerktunnel für zensurresistente Kommunikation

Entwurf, Implementierung und emulationsbasierte Evaluation

Bachelorarbeit

Vorgelegt von: Vorname Nachname
Matrikelnummer: 0000000

Erstgutachter: Prof. Dr. Erstgutachter
Zweitgutachter: Prof. Dr. Zweitgutachter

Abgabedatum: TT.MM.JJJJ

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Bachelorarbeit mit dem Titel “Zero-Knowledge-Netzwerktunnel für zensurresistente Kommunikation” selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Hamburg, den TT. MM. JJJJ

Vorname Nachname

Kurzfassung

Diese Arbeit entwirft und implementiert einen *Zero-Knowledge-Netzwerktunnel* für zensurresistente Kommunikation: ein providerblindes Relay, das nur Chifretext weiterleitet, während die Ende-zu-Ende-Verschlüsselung zwischen Client und Ursprung über das Noise-Protokoll erfolgt. Der Zugang zum Relay wird über ein Proof-of-Work-gestütztes Rendezvous und tokenbasierte Ratenbegrenzung geschützt. Der Prototyp ist in Rust auf Basis von QUIC umgesetzt und wird in einem Docker-basierten Testbett unter emulierten Netzbedingungen (`tc netem`) auf seine Roundtrip-Latenz hin evaluiert.

Abstract

This thesis designs and implements a *zero-knowledge network tunnel* for censorship-resistant communication: a provider-blind relay that forwards only ciphertext, while end-to-end encryption between client and origin is provided by the Noise protocol. Access to the relay is guarded by a proof-of-work rendezvous and per-token rate limiting. The prototype is implemented in Rust on top of QUIC and evaluated for round-trip latency in a Docker-based testbed under emulated network conditions (`tc netem`).

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Problemstellung	1
1.1.1	Vom Testbett zum selbst-hostbaren Produkt	2
1.2	Zielsetzung	2
1.3	Forschungsfragen	3
1.4	Aufbau der Arbeit	4
2	Grundlagen	5
2.1	Providerblinde Relays und das Zero-Knowledge-Prinzip	5
2.2	QUIC als Transport	6
2.2.1	Transportfallback über TCP	6
2.3	Das Noise-Protokoll	7
2.4	Proof-of-Work als Zugangsschutz	7
2.5	Capability- und Credential-Modell	8
2.6	Schlüsselrotation	8
2.7	Beobachtbarkeit unter Providerblindheit	9
2.8	Selbst-hostbare Steuerebene	9
3	Verwandte Arbeiten	11
3.1	Klassische VPNs und WireGuard	11
3.2	Reverse-Tunnel-Dienste	11
3.3	Anonymität und Zensurumgehung	12
3.4	QUIC-Relays und MASQUE	13
3.5	Abgrenzung	13
4	Anforderungen	15
4.1	Funktionale Anforderungen	15
4.2	Nichtfunktionale Anforderungen	16
4.3	Abgrenzung und Nicht-Ziele	17
4.4	Rückverfolgbarkeit	17
5	Architektur	19
5.1	Komponenten und Topologie	19
5.2	Schlüsselflüsse	20
5.3	Rollen-Dispatch am Edge	21
5.4	Verfügbarkeit: Redundanz, Failover und Reconnect	21
5.4.1	EdgeState-Registry und newest-first Failover	22
5.4.2	Erkennung toter Agenten	22
5.4.3	Reconnect-on-drop	22

5.5	Beobachtbarkeit	23
5.6	Vertrauensverteilung und Schlüsselrotation	24
5.6.1	Self-serve CA-Root-Verteilung	24
5.6.2	Zero-Downtime-Schlüsselrotation	24
5.7	Transport- und Applikationsflexibilität	24
5.7.1	QUIC-Primärtransport mit TLS-über-TCP-Rückfall	24
5.7.2	Cross-Transport-Relay	25
5.7.3	TLS-at-origin und Client-Forward-Modus	25
5.8	Ablauf des Verbindungsaufbaus	25
5.9	Ablauf des Failovers	26
5.10	Entwurfsentscheidungen	27
6	Implementierung	29
6.1	Aufbau des Workspace	29
6.2	Kryptographische Bausteine (ct-common)	29
6.3	Rollen-Dispatch am Edge (ct-edge)	31
6.4	Agent und Client	31
6.5	Agent-Redundanz und Failover (ct-edge)	32
6.6	Observability am Edge (ct-edge)	32
6.7	Selbstbediente Verteilung des CA-Roots (ct-control-plane)	33
6.8	Unterbrechungsfreie Schlüsselrotation (ct-agent)	33
6.9	Wiederverbinden des Agenten (ct-agent)	34
6.10	Transportübergreifendes Relay (ct-edge)	34
6.11	HTTPS am Origin und Client-Forward-Modus (ct-client)	34
6.12	Qualitätssicherung	35
6.13	Ausgewählte Implementierungsdetails	36
6.13.1	Die EdgeState-Registry: Typ, Failover, Eviction	36
6.13.2	Rendern des /metrics -Dokuments	37
6.13.3	Kopplung im Client-Forward-Modus	37
7	Evaluation	38
7.1	Testbett und Methodik	38
7.2	Grund-Overhead und Latenz-Skalierung	39
7.3	Funktionsexperimente	41
7.4	Beobachtbarkeit	42
7.5	Sicherheitsdiskussion und Bedrohungsmodell	43
7.6	Limitierungen und Validitätsbedrohungen	44
7.6.1	Interne und externe Validität im Überblick	45
7.7	Kosten des Proof-of-Work-Zugangsschutzes	46
7.7.1	Zerlegung des Fix-Overheads: Handschlag über der Propagation	47
7.8	Synthese der Forschungsfragen	48
8	Fazit und Ausblick	50
8.1	Zusammenfassung	50
8.2	Ausblick	52
8.3	Grenzen der Arbeit	53

1 Einleitung

1.1 Motivation und Problemstellung

Dienste hinter NAT oder Firewalls für entfernte Nutzer erreichbar zu machen, ist ein alltäglicher Bedarf; die verbreiteten Tunneldienste lösen ihn jedoch um den Preis des Vertrauens: Der Verkehr wird beim Anbieter terminiert, sodass dieser den Klartext einsehen und die Verbindung auf Zuruf kappen kann. Für Nutzer mit einem Schutzbedarf gegen Zensur ist ein Anbieter, der den Datenverkehr *sehen* oder *unterbrechen* kann, jedoch selbst ein Angriffspunkt.

Dieses Vertrauensproblem lässt sich anhand dreier Rollen präzisieren, die handelsübliche Tunneldienste in einer einzigen Instanz bündeln. Erstens terminiert der Anbieter die Transportverschlüsselung und liest damit die Nutzlast im Klartext; zweitens hält er die kryptographischen Identitäten (Zertifikate, Schlüssel) und ist damit die Wurzel des Vertrauens; drittens kontrolliert er den Datenpfad und kann einzelne Verbindungen selektiv unterbrechen. Wer gegen Zensur oder erzwungene Herausgabe geschützt sein will, muss jeder dieser drei Rollen misstrauen – und genau dieses Misstrauen ist bei einem terminierenden Anbieter technisch nicht durchsetzbar, sondern nur vertraglich zugesichert. Ein Betreiber, der den Klartext technisch nicht sehen kann, kann ihn auch unter Zwang nicht herausgeben; ein Betreiber, der die Schlüssel nicht hält, ist kein lohnendes Ziel für ihre Erpressung.

Diese Arbeit verfolgt den entgegengesetzten Entwurf: einen *providerblinden* (zero-knowledge) Tunnel, bei dem der Betreiber bewusst aus drei Dingen herausgehalten wird – dem *Inhalt* (die Nutzlast-Verschlüsselung terminiert erst am kundeneigenen Origin, nie beim Betreiber), dem *Vertrauen* (die Schlüssel sind kundenverankert, es gibt keine betreiberseitige PKI) und, soweit möglich, dem *Datenpfad* selbst. Die Zero-Knowledge-Eigenschaft ist dabei bewusst auf die *Nutzlast* begrenzt: Strukturelle Metadaten wie die Existenz eines Tunnels oder – im Relay-Fall – grobe Zeit- und Volumenzähler bleiben sichtbar. Durchsetzung reduziert sich auf einen einzigen, groben Hebel (Terminierung eines Tokens), der nur an einer eng definierten *rechtlichen Grenze* greift.

Damit diese Grenze nicht durch Umgehung des Anbieters ausgehebelt werden kann, verwendet der Entwurf einen *opaken Routing-Token* statt eines Hostnamens: Der Edge sieht weder eine SNI-Kennung noch einen DNS-Namen, sondern nur eine undurchsichtige Zeichenfolge, die eine Registrierung adressiert. Der Zugang zum Rendezvous wird durch einen *Proof-of-Work* (in der Tradition von Hashcash [2]) und eine Ratenbegrenzung pro Token verteuert, sodass ein

anonymer, betreiberblinder Dienst nicht trivial für Missbrauch geflutet werden kann. Die Ende-zu-Ende-Sicherheit ruht auf dem Noise-Protokoll-Rahmen [12], der Transport auf QUIC [9] mit TLS-1.3-Schlüsselverhandlung [17]; wo UDP blockiert ist, trägt ein TLS-über-TCP-Fallback denselben Tunnel.

Aus dieser Haltung ergibt sich ein Spannungsfeld: Providerblindheit und Zensurresistenz erkaufte man sich mit zusätzlichen Vermittlungsschritten (Rendezvous, Relay) und kryptographischem Aufwand. Ob und in welchem Umfang diese Schutzziele die Latenz eines Tunnels belasten, ist die zentrale empirische Frage dieser Arbeit.

1.1.1 Vom Testbett zum selbst-hostbaren Produkt

Ein providerblinder Tunnel ist als reines Forschungsartefakt wenig aussagekräftig: Erst der Anspruch, ihn tatsächlich *zu betreiben*, macht sichtbar, welche Eigenschaften über die Kryptographie hinaus notwendig sind. Die Arbeit betrachtet den Prototyp deshalb nicht nur als Latenz-Messobjekt, sondern als eine erste, selbst-hostbare Version 1 mit den Betriebseigenschaften, ohne die ein Zero-Knowledge-Design in der Praxis nicht tragfähig wäre:

- *Redundanz und Failover.* Ein einzelner Agent ist ein Ausfallpunkt zwischen Edge und Origin. Der Entwurf erlaubt es mehreren Agenten, sich unter derselben Origin-Identität zu registrieren; fällt der bedienende Agent aus, übernimmt nach einem Erkennungsfenster ein überlebender.
- *Beobachtbarkeit.* Ein betreiberblinder Dienst darf zwar keine Nutzlast einsehen, muss aber dennoch betreibbar sein. Der Edge exponiert daher ausschließlich *strukturelle* Kennzahlen (aktive Tunnel und Agenten, Registrierungs-, Relay- und Failover-Zähler) über einen Prometheus-Endpunkt, ohne die Zero-Knowledge-Grenze zu verletzen.
- *Selbst-Hosting.* Die dünne Kontrollebene hält keine Schlüssel und keine Nutzlast; sie lässt sich vom Kunden selbst betreiben und verteilt unter anderem die CA-Wurzel des Edge, sodass der Vertrauensanker nicht mehr außerhalb des Systems von Hand kopiert werden muss.

Diese Erweiterungen werden bewusst *ohne Überzeichnung* eingeführt: Der Prototyp bleibt eine Version 1 mit einer einzelnen Edge-Region und einem explizit gehaltenen Bedrohungsmodell (Abschnitt in Kapitel 7). Ziel ist nicht ein produktionsreifer Dienst, sondern der Nachweis, dass sich Providerblindheit und Betreibbarkeit im selben Entwurf vereinen lassen.

1.2 Zielsetzung

Ziel der Arbeit ist der Entwurf, die prototypische Implementierung und die emulationsbasierte Evaluation eines solchen Tunnels. Der Prototyp ist in Rust

auf Basis von QUIC umgesetzt und realisiert den providerblinden Relay-Pfad Client → Edge → Agent → Origin mit einem Proof-of-Work-gestützten, ratenbegrenzten Rendezvous. Die Noise-basierte Ende-zu-Ende-Verschlüsselung ist als kryptographischer Baustein umgesetzt. Die Bewertung erfolgt in einem vollständig Docker-basierten Testbett, in dem Netzbedingungen (Verzögerung, Paketverlust, Bandbreite) mit `tc netem` emuliert und die Roundtrip-Latenz reproduzierbar vermessen werden.

Über den reinen Latenzpfad hinaus umfasst die Umsetzung die Betriebseigenschaften aus Abschnitt 1.1.1: die Agenten-Redundanz mit Failover, die zustandslose Zero-Downtime-Rotation des Origin-Schlüssels bei unverändertem Routing-Token, den strukturellen Beobachtbarkeits-Endpunkt sowie die dünne, selbst-hostbare Kontrollebene. Der Umfang des Codes ist zugleich sein Qualitätsmaßstab: Das Rust-Workspace besteht aus fünf Crates, die durch 266 hermetisch laufende Unit- und Integrationstests mit einer bibliotheksbezogenen Zeilenabdeckung von 95,57 % abgesichert sind. Alle in dieser Arbeit berichteten Kennzahlen entstammen realen, reproduzierbaren Läufen; es werden keine Werte geschätzt oder extrapoliert.

1.3 Forschungsfragen

Die Arbeit gliedert sich an vier Forschungsfragen. FF1 bis FF3 betreffen den Kern des providerblinden Tunnels – seine Realisierbarkeit und sein Latenzverhalten; FF4 ergänzt die Frage nach der Betreibbarkeit, die aus dem Übergang vom Testbett zum selbst-hostbaren Produkt erwächst.

FF1 – Realisierbarkeit. Lässt sich ein providerblinder, zensurresistenter Tunnel aus etablierten Bausteinen – QUIC als Transport, Noise für die Ende-zu-Ende-Sicherheit, Proof-of-Work als Zugangsschutz – schlüssig entwerfen und implementieren?

FF2 – Latenz-Overhead. Welchen Grund-Overhead verursacht der providerblinde Vermittlungspfad (Verbindungsaufbau, PoW-Rendezvous, Mehrsprung-Relay) gegenüber einer direkten Verbindung?

FF3 – Verhalten unter Netzbedingungen. Wie skaliert die Tunnel-Roundtrip-Latenz mit Netzverzögerung und Paketverlust?

FF4 – Betreibbarkeit und Verfügbarkeit. Lassen sich die Betriebseigenschaften eines selbst-hostbaren Dienstes – Hochverfügbarkeit durch Agenten-Failover, strukturelle Beobachtbarkeit ohne Bruch der Zero-Knowledge-Grenze und Selbst-Hosting einer schlüssellosen Kontrollebene – innerhalb des providerblinden Entwurfs realisieren und funktional nachweisen?

1.4 Aufbau der Arbeit

Kapitel 3 grenzt den Entwurf gegen verwandte Systeme ab und Kapitel 4 leitet die funktionalen und nichtfunktionalen Anforderungen ab. Kapitel 2 führt die technischen Grundlagen ein (providerblinde Relays, das Noise-Protokoll, QUIC und der TLS-über-TCP-Fallback sowie Proof-of-Work als Zugangsschutz) und ordnet FF1 begrifflich ein. Kapitel 5 beschreibt den Systementwurf und seine Entwurfsentscheidungen – den providerblinden Datenpfad, die Zero-Knowledge-Grenze, die Agenten-Redundanz sowie die dünne, selbst-hostbare Kontrollebene – und legt damit die Grundlage für FF1 und FF4. Kapitel 6 dokumentiert die Umsetzung des Rust-Prototyps einschließlich der Beobachtbarkeit, der Schlüsselrotation und der Betriebswerkzeuge. Kapitel 7 stellt das Docker-Testbett und die Messmethodik vor, beantwortet FF2 und FF3 anhand der Latenzmatrix, weist die Betriebseigenschaften zu FF4 experimentell nach und diskutiert das Bedrohungsmodell und die Grenzen des Prototyps. Kapitel 8 fasst die Ergebnisse entlang der vier Forschungsfragen zusammen und gibt einen Ausblick auf die offenen Punkte, unter anderem die post-v1 zurückgestellte Browser-Ebene und die Mehr-Regionen-Erweiterung.

2 Grundlagen

Dieses Kapitel führt die technischen Bausteine ein, auf denen der Entwurf aufsetzt. Die ersten vier Abschnitte behandeln die tragenden Mechanismen des Datenpfades: das Prinzip providerblinder Relays (Abschnitt 2.1), QUIC als Transport (Abschnitt 2.2), das Noise-Protokoll für die Ende-zu-Ende-Sicherheit (Abschnitt 2.3) und Proof-of-Work als Zugangsschutz (Abschnitt 2.4). Die vier weiteren Abschnitte ergänzen die konzeptionellen Voraussetzungen, auf die spätere Kapitel zurückgreifen: das außerbandig verteilte Capability-Modell (Abschnitt 2.5), das Prinzip der Schlüsselrotation (Abschnitt 2.6), die für einen providerblinden Betreiber legitime Beobachtbarkeit (Abschnitt 5.5) und die selbst-hostbare, schlüsselfreie Steuerebene (Abschnitt 2.8). Alle Bausteine werden hier *konzeptionell* eingeführt; ihre konkrete Umsetzung und quantitative Bewertung folgen in den späteren Kapiteln.

2.1 Providerblinde Relays und das Zero-Knowledge-Prinzip

Ein *Relay* vermittelt Verkehr zwischen zwei Endpunkten, ohne selbst Endpunkt zu sein. Klassische Tunneldienste terminieren die Verschlüsselung beim Betreiber; dieser kann den Klartext lesen und ist damit sowohl ein Vertrauensanker als auch ein Angriffs- und Druckpunkt. Ein *providerblindes* Relay kehrt diese Eigenschaft um: Es leitet ausschließlich Chiffretext weiter und besitzt keinen Schlüssel, mit dem sich die Nutzlast entschlüsseln ließe. Die Verschlüsselung terminiert erst am kundeneigenen Ziel (*Origin*).

Die Zero-Knowledge-Eigenschaft ist dabei präzise auf die *Nutzlast* begrenzt. Der Betreiber kann weder Inhalt lesen oder verändern noch das Origin-Schlüsselmaterial erlangen; sichtbar bleiben ihm hingegen strukturelle *Metadaten*: dass ein Tunnel existiert, die Koordinationsereignisse des Rendezvous sowie – nur im Relay-Fall – grobe Zeit- und Volumenzähler. Adressiert wird ein Tunnel nicht über einen Hostnamen, sondern über ein opakes *Routing-Token*, sodass dem Betreiber auch das Vermittlungsziel verborgen bleibt. Diese Trennung von blinder Nutzlast und sichtbaren Metadaten ist der konzeptionelle Kern der Arbeit.

Es ist wichtig, die Blindheit nicht mit *Anonymität* zu verwechseln. Der Betreiber sieht weiterhin, dass *zwischen bestimmten Netzadressen* ein Tunnel besteht, und kann daran grobe Aktivitätsmuster ablesen; verborgen bleibt allein, *was* übertragen wird und *welchem* logischen Ziel das Routing-Token zugeordnet

ist. Damit richtet sich die Eigenschaft gegen zwei konkrete Bedrohungen: gegen einen kompromittierten oder unter Druck gesetzten Betreiber sowie gegen einen Angreifer, der sich Zugang zum Relay verschafft. In beiden Fällen liegt kein entschlüsselungsfähiges Material vor, das herausgegeben werden könnte. Der Entwurf formalisiert diese Trennung in zwei Architekturentscheidungen: einem providerblinden Ende-zu-Ende-Datenpfad (ADR-0001) und einer explizit gezogenen Zero-Knowledge-Grenze (ADR-0002), die festhält, welche Information den Betreiber legitim erreicht und welche ihn niemals erreichen darf.

2.2 QUIC als Transport

QUIC ist ein UDP-basiertes, gesichertes Transportprotokoll [9], dessen Sicherheitsschicht auf TLS 1.3 aufsetzt [17]. Für einen Tunnel ist es aus mehreren Gründen geeignet: Es multiplext mehrere Ströme über eine Verbindung ohne wechselseitiges Head-of-Line-Blocking unter Paketverlust, es unterstützt Verbindungsmigration bei wechselnden IP-Adressen und es erlaubt einen schnellen Verbindungsaufbau. Im hier verfolgten Entwurf baut der Agent die Verbindung *ausgehend* zum Edge auf, sodass auf der Kundenseite keine eingehenden Ports geöffnet werden müssen; eine Client-Verbindung bildet sich auf einen QUIC-Strom ab. Die innere, providerblinde Sitzung (Abschnitt 2.3) läuft byteweise über diesen Transport und ist von der QUIC-eigenen TLS-Sicherung des Agent↔Edge-Abschnitts unabhängig.

2.2.1 Transportfallback über TCP

Der ausgehende UDP-Verkehr, auf dem QUIC beruht, ist in restriktiven Netzen nicht immer verfügbar: Firmen-Firewalls, Captive Portals und einzelne Mobilfunknetze blockieren UDP/443 oder drosseln es so stark, dass ein Verbindungsaufbau scheitert. Damit der Dienst auch dort nutzbar bleibt, sieht der Entwurf einen Rückfall auf einen TLS-Strom über TCP/443 vor (ADR-0004). Dieser Pfad tarnt sich gegenüber einem einfachen Portfilter wie gewöhnlicher HTTPS-Verkehr und trägt denselben inneren Tunnel: Die providerblinde Noise-Sitzung ist gegenüber dem Untenliegenden agnostisch und läuft byteorientiert über QUIC *oder* TCP, ohne dass sich an der Ende-zu-Ende-Sicherheit etwas ändert. Der Transport ist damit eine austauschbare Schicht; die für den Kunden relevante Sicherheitsaussage ist von der Wahl QUIC gegenüber TCP entkoppelt. Der Preis des Fallbacks ist der Verlust der QUIC-spezifischen Vorteile – insbesondere wird das Head-of-Line-Blocking von TCP unter Paketverlust wieder wirksam –, weshalb QUIC der bevorzugte und TCP der subsidiäre Weg bleibt.

2.3 Das Noise-Protokoll

Die Ende-zu-Ende-Sicherheit zwischen Client und Origin stellt das Noise-Protokoll-Framework [12] her – konkret ein Muster vom Typ `Noise_IK` mit statischen X25519-Schlüsselpaaren, symmetrischer Verschlüsselung über ChaCha20-Poly1305 und BLAKE2s als Hashfunktion. Der Client kennt und *pinnt* den statischen öffentlichen Schlüssel des Origin (die *Origin Identity*); beide Seiten authentisieren sich über ihre statischen Schlüssel. Das Ergebnis ist ein gegenseitig authentisierter, vorwärtsgeheimer Kanal *ohne* Zertifizierungsstelle oder PKI.

Die Wahl des Musters `Noise_IK` ist bewusst: Das Kürzel “I” bezeichnet die *immediate*, aber verschlüsselt übertragene statische Identität des Initiators, “K” die dem Initiator vorab *known* (bekannte) statische Identität des Responders. Genau diese Vorabkenntnis passt zum Bedrohungsmodell: Der Client soll ausschließlich mit dem gepinnten Origin sprechen und nicht auf eine vom Betreiber untergeschobene Identität hereinfallen. Da der Responder-Schlüssel bereits vor dem ersten Paket feststeht, kann der Handshake in einem Umlauf abgeschlossen werden.

Gerade das Fehlen eines externen Ausstellers ist für die Zensurreistenz wesentlich: Es gibt keine Instanz, die zur Sperrung oder zum Ausstellen falscher Identitäten gedrängt werden könnte. Die Noise-Sitzung ist ein *innerer* Handshake über den Transportstrom und damit unabhängig von der Transportsicherung. Der Preis dieser Entkopplung ist, dass die Schlüsselverteilung in der Verantwortung des Kunden liegt: Clients müssen die Origin Identity außerhalb des Bandes erhalten (Abschnitt 2.5), und der Schlüsseltausch ist eine explizite Operation ohne Ablauf- oder Erneuerungsautomatik (Abschnitt 2.6).

2.4 Proof-of-Work als Zugangsschutz

Da der Betreiber providerblind ist und bewusst auf WAF, Abuse-Feeds und Identitätsprüfung (KYC) verzichtet, fehlen ihm die üblichen Hebel gegen Denial-of-Service und Sybil-Angriffe. Als Ersatz dient ein gestaffeltes Schema, dessen Kern ein *Proof-of-Work* (PoW) ist [2]: Teure Operationen – eine Rendezvous-Anfrage oder das Belegen eines Relay-Platzes – erfordern den Nachweis einer verrichteten Rechenarbeit, typischerweise das Finden einer Nonce, deren Hash eine geforderte Schwierigkeit erfüllt. Die Prüfung des Nachweises ist für den Betreiber billig, seine Erzeugung für den Anfragenden teuer; diese Asymmetrie verteuert das Fluten, ohne einzelne legitime Aufbauten spürbar zu belasten.

Ergänzt wird der Nachweis durch *Ratenbegrenzung* pro Token sowie durch Kapazitätsreserven. Der PoW ist zugleich der primäre Sybil-Schutz in Abwesenheit von KYC. Seine Grenze ist bekannt und wird in dieser Arbeit nicht ausgeblendet: Die Schwierigkeit muss Fluten abschrecken, ohne legitime Nutzung zu behindern, und ein hinreichend finanzierter Angreifer kann die Kosten schlicht bezahlen – missbrauchs- und abrechnungsseitige Sybil-Resistenz bleibt

ein offener Punkt (Kapitel 8). Die Schwierigkeit ist ein Betriebsparameter und gehört damit zu den in der Evaluation variierten Größen; sie geht als fester Zusatzaufwand in die Latenz des Tunnelaufbaus ein. Der Zusammenhang von PoW-Kosten, Aufbau Latenz und Blind-DoS-Resistenz wird im Entwurf als eigene Entscheidung geführt (ADR-0018) und in der Auswertung experimentell beleuchtet.

2.5 Capability- und Credential-Modell

Weil weder Betreiber noch eine PKI als Vertrauensanker dienen, muss ein Client alles, was er für die Kontaktaufnahme mit einem bestimmten Origin braucht, in gebündelter Form erhalten. Diese Bündelung nennt der Entwurf eine *Capability*: ein selbsttragendes Berechtigungsobjekt, das mindestens das opake Routing-Token (zum Adressieren des Tunnels beim Betreiber) und die zu pinnende Origin Identity (zur Ende-zu-Ende-Authentisierung im Noise-Handshake) zusammenfasst. Wer eine gültige Capability besitzt, kann den zugehörigen Tunnel nutzen; wer sie nicht besitzt, kann das Ziel nicht einmal benennen, da anstelle eines Hostnamens nur das opake Token existiert.

Entscheidend ist der Verteilweg: Capabilities werden *außerhalb des Bandes* übergeben (ADR-0014), also nicht über den Betreiber und nicht über den Datenpfad, den sie erst erschließen. Der Kunde entscheidet selbst, über welchen vertrauenswürdigen Kanal er sie an berechnete Clients ausliefert. Diese Entkopplung ist die logische Konsequenz der Providerblindheit: Ginge die Berechtigung durch die Hände des Betreibers, wäre dieser wieder ein Vertrauensanker und ein Druckpunkt. Der bewusste Verzicht auf einen betreibergestützten Verteildienst verschiebt die Verantwortung für die Schlüssel- und Berechtigungsverteilung damit vollständig zum Kunden – ein Nachteil an Komfort, der im Gegenzug die Zero-Knowledge-Grenze wahrt. Eine eng umrissene Ausnahme betrifft nicht-geheimes Material: Die *öffentliche* CA-Wurzel des Edge kann über die Steuerebene bereitgestellt werden (Abschnitt 2.8), da ihre Kenntnis keine Berechtigung verleiht.

2.6 Schlüsselrotation

Da die Origin Identity nicht abläuft, muss ein Betreiberwechsel des Schlüsselmaterials – etwa bei Verdacht auf Kompromittierung oder als Routine – als explizite Operation möglich sein, und zwar ohne den laufenden Betrieb zu unterbrechen. Das tragende Prinzip lautet: *gleiches Routing-Token, neue Origin-Identität*. Die Rotation trennt die beiden Bestandteile einer Capability sauber voneinander. Das Routing-Token, über das der Betreiber vermittelt, bleibt unverändert – der Tunnel behält seine Adresse, und bereits verteilte Clients routen weiterhin korrekt. Erneuert wird allein das kryptographische Origin-Schlüsselpaar, das der Noise-Handshake authentisiert.

Damit während der Umstellung keine Verbindung abreißt, hält der Origin für ein begrenztes *Rotationsfenster* sowohl den zurückgezogenen als auch den neuen Schlüssel bereit. Ein Client mit einer alten Capability handshaked in diesem Fenster noch gegen den retirierten Schlüssel, ein Client mit einer bereits aktualisierten Capability gegen den neuen – beide erreichen dasselbe Origin. Nach Ablauf des Fensters wird der alte Schlüssel entfernt, und nur noch die neue Origin Identity ist gültig. So entsteht eine unterbrechungsfreie Rotation, bei der die Verteilung der *neuen* Origin Identity an die Clients wiederum über den außerbandigen Capability-Kanal (Abschnitt 2.5) erfolgt. Weil das Routing-Token stabil bleibt, erfährt der Betreiber von der Rotation nichts; für ihn ist der Tunnel schlicht durchgehend erreichbar. Die praktische Wirksamkeit dieses Verfahrens wird in der Evaluation über ein Rotationsexperiment belegt.

2.7 Beobachtbarkeit unter Providerblindheit

Auch ein providerblinder Betreiber muss seinen Dienst betreiben können: Er benötigt Kennzahlen zu Auslastung, Fehlern und Kapazität. Die Kernfrage lautet, *welche* Beobachtung mit der Zero-Knowledge-Grenze verträglich ist. Der Leitgedanke: Erlaubt ist ausschließlich *strukturelle* Telemetrie, die sich aus dem legitim sichtbaren Metadatenanteil (Abschnitt 2.1) speist und keinerlei Rückschluss auf Nutzlast, Origin-Ziel oder Schlüsselmaterial gestattet.

Der Entwurf exponiert diese Telemetrie im etablierten Prometheus-Textformat: Zeitreihen, die durch periodisches Abfragen (*Scraping*) eines `/metrics`-Endpunkts eingesammelt werden. Jede Zeile trägt einen Metriknamen, optionale Beschriftungen (*Labels*) und einen Zahlenwert; *Gauges* bilden momentane Zustände ab (etwa die Zahl aktiver Tunnel oder registrierter Agenten), *Counter* kumulieren monoton wachsende Ereignisse (etwa die Gesamtzahl an Registrierungen, Relay-Vorgängen, relayten Bytes oder Failover-Ereignissen). Charakteristisch für die providerblinde Ausprägung ist, dass diese Reihen bewusst *grob* bleiben: Ein summierter Byte-Zähler misst Volumen, ohne den Inhalt zu berühren; ein Tunnelzähler belegt Existenz, ohne ein Ziel zu benennen. Auf inhaltsnahe oder pro-Ziel aufgelöste Beschriftungen wird verzichtet, da sie die Metadatenparsamkeit unterlaufen würden. Ergänzend hält die Steuerebene eine aggregierte Statussicht bereit, deren Tunnelzahl aus genau diesen Edge-Zählern gespeist wird und nicht aus einer separaten, feiner auflösenden Registratur (ADR-0016). Damit wird Beobachtbarkeit zu einem first-class-Entwurfsziel, das dieselbe Grenze respektiert wie der Datenpfad.

2.8 Selbst-hostbare Steuerebene

Der Datenpfad braucht eine *Steuerebene* für Aufgaben, die nicht sicherheitskritisch am Chiffretext hängen: Kontenverwaltung, Guthaben- bzw. Zahlungsnachweise, das Veröffentlichen der öffentlichen CA-Wurzel und eine aggregier-

2 Grundlagen

te Statussicht. Der Entwurf hält diese Steuerebene bewusst *dünn* und *selbst-hostbar* (ADR-0017): Sie hält *keine* Schlüssel und *keine* Nutzlast und liegt damit vollständig außerhalb der Zero-Knowledge-Grenze. Fiele sie aus oder würde sie kompromittiert, blieben Vertraulichkeit und Integrität der Tunnel unberührt, weil beide allein am Ende-zu-Ende-Noise-Kanal hängen.

Die *Selbst-Hostbarkeit* ist dabei mehr als ein Betriebsmerkmal: Sie verhindert, dass ein zentraler, notwendig vertrauenswürdiger Dienst zum Single Point of Trust und damit zum Zensur- oder Druckpunkt wird. Wer dem Anbieter der Steuerebene nicht vertrauen will, betreibt sie selbst; die providerblinde Sicherheitsaussage des Datenpfades ändert sich dadurch nicht. Für den regulären, gehosteten Betrieb bindet die Steuerebene die Kontenverwaltung an eine externe Identitätsinstanz (OIDC bzw. Keycloak) an, über die nutzerbezogene Endpunkte authentisiert werden, und führt ein Guthaben-/Kredit-Ledger für die abrechnungsseitige Zugangssteuerung. Diese Funktionen sind vom Datenpfad entkoppelt: Der Edge relayt Chiffretext auch dann korrekt, wenn die Steuerebene nichts über den Inhalt oder das Ziel eines Tunnels weiß. Die klare Zweiteilung – schlüsselfreie, ersetzbare Steuerebene neben providerblindem Datenpfad – ist die organisatorische Entsprechung der technischen Zero-Knowledge-Grenze und wird in den folgenden Kapiteln zum Architekturprinzip ausgebaut.

3 Verwandte Arbeiten

Dieses Kapitel ordnet `claude-tunnel` in bestehende Systeme ein und schärft die zentrale Unterscheidung der Arbeit heraus: die *Providerblindheit*. Betrachtet werden vier Familien verwandter Systeme — klassische VPNs (Abschnitt 3.1), Reverse-Tunnel-Dienste (Abschnitt 3.2), Anonymitäts- und Zensurumgehungssysteme (Abschnitt 3.3) sowie QUIC-basierte Relays (Abschnitt 3.4). Der abschließende Abschnitt 3.5 verdichtet die Abgrenzung in einer Gegenüberstellung. Leitfrage ist durchgehend: *Wer terminiert die Verschlüsselung, und wer kann folglich Klartext lesen oder die Verbindung kappen?*

3.1 Klassische VPNs und WireGuard

Klassische VPNs stellen einen verschlüsselten Tunnel zwischen zwei Endpunkten her. Bei WireGuard [5] etwa terminiert die Verschlüsselung an den beiden Peers selbst: Der Tunnel verbindet einen Client mit einem Server, der zugleich Gegenstelle und Vertrauensanker ist. Für den hier verfolgten Anwendungsfall — ein kundeneigenes Origin hinter einer restriktiven Netzgrenze ohne öffentliche Adresse erreichbar zu machen — ist dieses Modell nur bedingt geeignet: Es existiert kein providerblindes *Relay* zwischen den Parteien, das Verkehr vermittelt, ohne selbst Endpunkt zu sein. Wer einen VPN-Server als Vermittlungsknoten betreibt, terminiert dessen Verschlüsselung und sieht den weitergeleiteten Verkehr im Klartext. WireGuard löst die Transport- und Schlüsselfrage elegant, adressiert aber gerade nicht die Trennung von blinder Vermittlung und kundenseitig verankerter Ende-zu-Ende-Sicherheit, die den Kern dieser Arbeit bildet. `claude-tunnel` übernimmt die Idee moderner, schlanker Handshakes — die Datenebene nutzt Noise [12] —, verschiebt die Terminierung jedoch vom Vermittler an das kundeneigene Ziel.

3.2 Reverse-Tunnel-Dienste

Reverse-Tunnel-Dienste wie Cloudflare Tunnel [3], ngrok [10] oder Tailscale Funnel [16] lösen dieselbe Erreichbarkeitsaufgabe: Ein Agent im privaten Netz baut eine ausgehende Verbindung zu einer Anbieter-Infrastruktur auf, über die eingehender Verkehr an das interne Ziel zurückgereicht wird. Damit entfällt die

Notwendigkeit eingehender Firewall-Freigaben. Entscheidend für die Abgrenzung ist jedoch, *wo die Transportverschlüsselung terminiert*: Bei diesen Diensten endet die TLS-Sitzung beim Anbieter. Der Betreiber steht als vollwertiger Man-in-the-Middle im Pfad, sieht den Klartext des vermittelten Verkehrs und kann einzelne Tunnel jederzeit inhaltlich inspizieren oder kappen. Adressiert werden die Tunnel überdies typischerweise über öffentliche Hostnamen, deren Zuordnung der Anbieter kennt und kontrolliert.

Genau dieses Vertrauensmodell ist das *Gegenmodell* zu `claude-tunnel`. Der providerblinde Ansatz kehrt die Eigenschaft um: Das Relay leitet ausschließlich Chiffretext weiter, hält keinen Schlüssel zur Nutzlast und adressiert Tunnel nicht über einen Hostnamen, sondern über ein opakes *Routing-Token* ohne SNI-Information. Der Betreiber bleibt damit von Inhalt und Vermittlungsziel abgeschnitten und ist als Druck- oder Zensurpunkt entwertet — die funktionale Bequemlichkeit des Reverse-Tunnels wird ohne die Vertrauensbürde des klartextsichtigen Anbieters erreicht.

3.3 Anonymität und Zensurumgehung

Tor [4] und allgemeiner das Onion-Routing verfolgen ein anderes Bedrohungsmodell: Ziel ist die *Anonymität* der kommunizierenden Parteien, erreicht durch mehrstufige, geschichtete Verschlüsselung über mehrere unabhängige Relays, sodass kein einzelner Knoten Sender und Empfänger zugleich kennt. *Pluggable transports* wie obfs4 [20] oder Shadowsocks [15] ergänzen dies um Verschleierung des Verkehrsmusters, um Deep-Packet-Inspection und Protokollfilter zu unterlaufen. Weitere Umgehungsansätze verlagern die Resistenz in die Netzinfrastruktur — etwa Domain Fronting [7], das Sperren durch Mitnutzung großer, nicht blockierbarer Fronting-Domains unterläuft, oder Refraction-/Decoy-Routing wie Telex [19]; Messplattformen wie OONI [8] dokumentieren Verbreitung und Methodik solcher Blockaden empirisch. Diese Systeme schützen also primär die *Identität* und die *Beobachtbarkeit* der Kommunikation gegenüber Netzbeobachtern.

`claude-tunnel` zielt demgegenüber nicht auf Anonymität, sondern auf *Providerblindheit*: Der Betreiber darf durchaus wissen, *dass* ein Tunnel existiert und grobe Zeit- und Volumenzähler sehen — er darf nur die Nutzlast weder lesen noch das Origin-Schlüsselmaterial erlangen. Die Bedrohungsmodelle sind damit komplementär, nicht deckungsgleich. Verwandt bleiben gleichwohl die Fragen der *Metadaten*-Minimierung (welche strukturellen Spuren ein Vermittler zwangsläufig sieht) und der *Denial-of-Service*-Resistenz eines offenen, KYC-freien Zugangs. Wo Tor auf Netzwerk- und Reputationsmechanismen setzt, verwendet `claude-tunnel` ein Proof-of-Work-Rendezvous [2] mit tokenweiser Ratenbegrenzung als primären Sybil- [6] und Flutungs-Schutz.

Tabelle 3.1: Abgrenzung entlang von Terminierung, Klartextsicht, Kappbarkeit und Adressierung.

Eigenschaft	WireGuard	Cloudflare Tunnel	Tor
Wer terminiert TLS/Krypto	Peer/Server	Anbieter	Origin (mehrfach)
Wer sieht Klartext	Server-Peer	Anbieter	kein Einzelknoten
Wer kann kappen	Server-Peer	Anbieter	jedes Relay (Pfad)
Öffentl. Hostname nötig	Server-Endpunkt	ja	nein

3.4 QUIC-Relays und MASQUE

Auf der Transportebene ist `claude-tunnel` den QUIC-basierten Relay-Ansätzen verwandt. QUIC [9] bündelt Verbindungsaufbau, Verschlüsselung und Multiplexing über UDP und eignet sich damit für den schnellen Aufbau vieler kurzlebiger Tunnel; die Vertraulichkeit ruht dabei auf TLS 1.3 [13], das QUIC in seinen Handshake integriert. Das MASQUE-Framework [14, 11] baut auf QUIC und HTTP/3 auf, um Proxy- und Tunneldienste über eine QUIC-Verbindung zu transportieren — konzeptionell ein Relay, das UDP- oder IP-Verkehr für einen Client weiterreicht. Die Datenebene dieser Arbeit teilt diese Transportbasis: QUIC (via `quinn`) ist der primäre Transport, ergänzt um einen TLS-über-TCP-Rückfall für Umgebungen, in denen UDP blockiert ist.

Der Unterschied liegt erneut in der Terminierung und der Schlüsselverankerung: Ein MASQUE-Proxy terminiert die QUIC-Verbindung des Clients bei sich und ist somit prinzipiell klartextsichtig auf der Proxy-Sitzung. `claude-tunnel` legt über den QUIC-Transport eine zweite, providerblinde Schicht: eine Ende-zu-Ende-Noise-Sitzung zwischen Client und Origin, deren Chiffretext das Relay lediglich durchreicht. Der Transport ist verwandt; das Vertrauensmodell ist es nicht.

3.5 Abgrenzung

Tabelle 3.1 stellt die vier Familien anhand der entscheidenden Merkmale gegenüber. Der Unterschied bündelt sich in einer Zeile: Bei WireGuard, Cloudflare Tunnel und Tor liegt die für den Nutzlastschutz relevante Terminierung entweder beim Vermittler oder ist gar nicht als blindes Relay konzipiert; nur `claude-tunnel` verbindet ein nutzlast-blindes Relay mit einer kundenseitig am Origin verankerten Terminierung.

Die konzeptionell nächsten Vorläufer sind nicht die klassischen Tunnel, sondern jüngere Ansätze, die das *Wissen* über eine Verbindung bewusst auf mehrere Parteien aufteilen. Oblivious HTTP [18] trennt den Anfrageinhalt von der Absenderidentität: Ein Relay verbirgt die Client-Adresse, während ein Gateway die Anfrage entschlüsselt und an das Ziel weiterreicht. Damit sieht der

3 Verwandte Arbeiten

Gateway sehr wohl Ziel und Klartext, und das Modell ist auf einzelne, zustandslose HTTP-Anfragen zugeschnitten statt auf einen langlebigen, protokollagnostischen Tunnel. Apples iCloud Private Relay [1] nutzt zwei Hops mit Ingress-/Egress-Trennung, sodass kein einzelner Betreiber zugleich Client-IP und Zieladresse kennt; die Trennung ruht jedoch auf *zwei getrennten Betreibern*, und die Sitzung zum Ziel terminiert regulär — eine kundenseitig verankerte Nutzlast-Blindheit gegenüber einem *einzelnen* Vermittler ist damit nicht gegeben. `claude-tunnel` grenzt sich gegen beide dadurch ab, dass ein *einzelner* Betreiber weder Nutzlast noch Vermittlungsziel erfährt — Chiffretext statt Klartext, opakes Routing-Token statt Hostname — und die Terminierung unabhängig von einer Zwei-Parteien-Aufteilung am kundeneigenen Origin verankert bleibt.

Die Alleinstellung von `claude-tunnel` ergibt sich aus der Kombination vierer Bausteine, die in dieser Zusammenstellung bei keinem der verglichenen Systeme — auch nicht bei diesen nächsten Vorläufern — auftritt:

- ein *nutzlast-blindes Relay*, das ausschließlich Chiffretext weiterleitet und keinen Schlüssel zur Nutzlast besitzt;
- ein *opakes Routing-Token* statt eines Hostnamens, das dem Betreiber auch das Vermittlungsziel verbirgt (keine SNI- oder Hostnamen-Information);
- ein *Proof-of-Work-Rendezvous* mit tokenweiser Ratenbegrenzung als KYC-freier Zugangs- und DoS-Schutz [2];
- *kundenseitig verankerte Noise-Schlüssel ohne PKI* [12] — die Ende-zu-Ende-Sicherheit terminiert am kundeneigenen Origin und benötigt weder eine Zertifikathierarchie noch einen anbieterseitigen Vertrauensanker.

Für `claude-tunnel` bedeutet dies: Die funktionale Erreichbarkeit eines Reverse-Tunnels wird mit dem Vertrauensmodell eines providerblinden Relays zusammengeführt — der Betreiber vermittelt, ohne Endpunkt oder Vertrauensanker zu sein.

4 Anforderungen

Aus der Motivation, dem Bedrohungsmodell und den Forschungsfragen leiten sich die funktionalen und nichtfunktionalen Anforderungen an `claudetunnel` ab. Jede Anforderung wird nachfolgend benannt, begründet und auf ein tatsächlich realisiertes Merkmal des Systems zurückgeführt. Die Rückverfolgbarkeit fasst Tabelle 4.1 zusammen; sie verweist auf die real durchgeführten Experimente und die zugehörigen Artefakte.

4.1 Funktionale Anforderungen

- FA1 — Providerblinder Relay-Pfad** Nutzdaten müssen entlang der Kette Client → Edge → Agent → Origin transportiert werden, ohne dass der Edge Klartext oder Origin-Schlüssel erhält. Realisiert durch die providerblinde Datenebene (ADR-0001/0002); der Edge relayt ausschließlich Noise-Chiffprat.
- FA2 — PoW-gestütztes, ratenbegrenztes Rendezvous** Der Zugang zum Rendezvous muss durch einen Proof-of-Work (PoW) mit einstellbarer Schwierigkeit und eine Ratenbegrenzung pro Routing-Token geschützt sein, um Fluten zu erschweren.
- FA3 — Selbsttragende Capabilities** Berechtigungen müssen out-of-band als selbsttragende Capabilities übergeben werden (ADR-0014); ein opakes Routing-Token ersetzt Hostnamen bzw. SNI.
- FA4 — Agenten-Redundanz und Failover** Mehrere Agenten müssen sich unter einer Origin-Identität registrieren; fällt der bedienende Agent aus, muss das Routing auf eine überlebende Registrierung umschalten. Realisiert über die Edge-Registry (`HashMap<RoutingToken, Vec<(id, Connection)>>`) mit *newest-first*-Failover und Eviction toter Verbindungen via `keep_alive 3s` und `max_idle_timeout 10s`.
- FA5 — Beobachtbarkeit** Der Betreiber muss den Systemzustand über `/metrics` (Prometheus-Gauges und -Counter am Edge) und `/status` (Kontrollebene) einsehen können, inklusive einer selbsttragenden HTML-Landingpage.
- FA6 — Zero-Downtime-Schlüsselrotation** Der Origin-Schlüssel muss unter Beibehaltung desselben Routing-Tokens rotiert werden können; während eines Fensters müssen alte und neue Capability weiterhin einen Tunnel aufbauen.

- FA7 — QUIC primär mit TLS-über-TCP-Fallback** Der Datenpfad muss primär über QUIC laufen und bei blockiertem UDP transparent auf TLS-über-TCP (Port 443) zurückfallen (ADR-0004), ohne die Tunnelsemantik zu ändern.
- FA8 — Cross-Transport-Relay** Ein QUIC-Client muss einen Agenten erreichen können, der auf dem TCP-Fallback läuft (#13); der Edge vermittelt zwischen den Transporten.
- FA9 — Self-hostbare Steuerebene** Die Kontrollebene muss dünn und selbst-hostbar sein (ADR-0017), keine Schlüssel oder Nutzdaten halten und OIDC/Keycloak-verifizierte Endpunkte sowie ein Konto-/Kredit-Ledger bereitstellen.
- FA10 — TLS-at-origin durch den Tunnel** HTTPS-Endpunkte müssen durch den Tunnel erreichbar sein, wobei TLS am Origin terminiert und die Zertifikatsprüfung client-seitig erfolgt; der Edge sieht nur TLS-über-Noise-Chiffre.

4.2 Nichtfunktionale Anforderungen

- NFA1 — Zero-Knowledge / Providerblindheit** Der Edge darf strukturell nur Metadaten (Existenz eines Tunnels, grobe Zeit-/Volumenzähler, Rendezvous-Ereignisse) beobachten, niemals Nutzdaten oder Schlüssel (ADR-0001/0002).
- NFA2 — Kleiner, vorhersagbarer Latenz-Overhead** Der durch Tunnelaufbau (QUIC + PoW + Noise_IK + Relay) verursachte Zusatzaufwand soll gering und über Netzbedingungen hinweg vorhersagbar bleiben. Belegt durch die Latenzmatrix (`scripts/sweep.sh`), in der der mittlere Round-Trip bei 0% Verlust nahezu linear mit der emulierten Verzögerung skaliert.
- NFA3 — Verfügbarkeit / Hochverfügbarkeit** Der Dienst muss den Ausfall eines Agenten überstehen (Failover, vgl. FA4), ohne dass der Client den Tunnel manuell neu aufbauen muss.
- NFA4 — Betreibbarkeit und Beobachtbarkeit** Der Betrieb muss ohne Instrumentierung durch Dritte diagnostizierbar sein; Zähler und Statusseite müssen ausreichen, um Registrierungen, Relays und Failover nachzuvollziehen (ADR-0016).
- NFA5 — Sicherheit ohne PKI-Zwang** Die Ende-zu-Ende-Vertraulichkeit muss auf Noise_IK (X25519/ChaChaPoly/BLAKE2s) beruhen und ohne öffentliche PKI auskommen; Vertrauen wird über die out-of-band verteilte Capability bzw. die self-serve CA-Root-Verteilung (#11) hergestellt.

NFA6 — DoS-/Sybil-Widerstand ohne KYC Das System muss Fluten ohne Identitätsprüfung erschweren; der PoW ist der primäre Sybil-/DoS-Schutz (ADR-0018). Der PoW-Schwierigkeitsknopf muss so einstellbar sein, dass er Fluten abschreckt, ohne legitime Clients zu bestrafen.

4.3 Abgrenzung und Nicht-Ziele

- **Browser-Ebene zurückgestellt.** Ein öffentlich adressierbarer Browser-Pfad mit öffentlichem SNI und Let's-Encrypt-Zertifikaten ist *nicht* Ziel von v1; er ist post-v1 zurückgestellt (ADR-0010, Issue #23). v1 belegt stattdessen, dass TLS am Origin terminiert (FA10).
- **Anonymität ist kein Ziel.** `claude-tunnel` stellt Providerblindheit gegenüber dem Edge her, jedoch keine Anonymität der Endpunkte im Sinne eines Anonymisierungsnetzes; Verkehrsflussanalyse durch globale Beobachter ist ausdrücklich außerhalb des Bedrohungsmodells.
- **Schlüsselverteilung liegt beim Kunden.** Die Verteilung der Capabilities erfolgt out-of-band (ADR-0014); Rotation ist explizit und ohne automatische Ablauflogik (FA6).

4.4 Rückverfolgbarkeit

Tabelle 4.1 führt jede Anforderung auf das realisierende Merkmal und das belegende Experiment zurück. Alle genannten Experimente wurden real ausgeführt; die zugehörigen Skripte, Zähler und Zeitreihen sind im Repository hinterlegt.

4 Anforderungen

Tabelle 4.1: Rückverfolgbarkeit: Anforderung → realisierendes Merkmal / Experiment.

Anforderung	Realisierendes Merkmal	Beleg / Experiment
FA1	Providerblinder Datenpfad (ADR-0001/0002)	<code>e2e-smoke.sh</code> (#6)
FA2	PoW + Ratenbegrenzung pro Token	PoW-Studie <code>sweep.sh</code>
FA3	Out-of-band Capabilities (ADR-0014)	opakes Routing-Token
FA4	Edge-Registry + Failover	<code>redundancy-smoke.sh</code> (#8)
FA5	<code>/metrics</code> + <code>/status</code>	Metrik-Scrape (#10)
FA6	Origin-Schlüsselrotation	<code>rotation-smoke.sh</code> (#12)
FA7	QUIC + TLS-über-TCP-Fallback (ADR-0004)	<code>via=quic/tcp</code> (#6)
FA8	Cross-Transport-Relay	QUIC-Client zu TCP-Agent
FA9	Dünne self-hostbare Steuerebene (ADR-0017)	<code>/status</code> , OIDC (#10)
FA10	TLS-at-origin durch den Tunnel	<code>https-demo.sh</code> (#22)
NFA1	Zero-Knowledge-Grenze (ADR-0002)	nur strukturelle Metadaten
NFA2	Latenz-Overhead klein/vorhersagbar	Latenzmatrix <code>sweep.sh</code>
NFA3	HA / Failover	<code>redundancy-smoke.sh</code> (#8)
NFA4	Betreibbarkeit / Beobachtbarkeit	Zähler + Landingpage (#4, #
NFA5	Noise_IK, kein PKI-Zwang	CA-Root-Verteilung (#11)
NFA6	PoW-Sybil-/DoS-Schutz (ADR-0018)	PoW-Studie <code>sweep.sh</code>

5 Architektur

Dieses Kapitel beschreibt den Systementwurf: die Komponenten und ihr Zusammenspiel (Abschnitt 5.1), die tragenden Flüsse (Abschnitt 5.2), das am Edge implementierte Rollen-Dispatch (Abschnitt 5.3), die Verfügbarkeits- und Resilienzmechanik (Abschnitt 5.4), die Beobachtbarkeit (Abschnitt 5.5), die self-serve Vertrauensverteilung und Schlüsselrotation (Abschnitt 5.6), die Transport- und Applikationsflexibilität (Abschnitt 5.7) und schließlich die wesentlichen Entwurfsentscheidungen (Abschnitt 5.10). Der Entwurf ist durchgängig *providerblind*: Der öffentlich erreichbare Knoten sieht zu keinem Zeitpunkt Klartext, Schlüsselmaterial oder Zielidentitäten, sondern ausschließlich strukturelle Metadaten. Diese Leitentscheidung (ADR-0001, ADR-0002) prägt jede der folgenden Komponenten.

5.1 Komponenten und Topologie

Der Entwurf kennt vier Datenpfad-Rollen und eine Steuerkomponente (Abbildung 5.1):

Origin. Der private Dienst, den der Kunde exponieren will. Hier terminiert die Verschlüsselung; das Origin-Schlüsselmaterial verlässt diesen Bereich nie. Der Origin kann ein einfacher TCP-Dienst sein oder — wie in Abschnitt 5.7 gezeigt — selbst TLS terminieren, sodass die Zertifikatsvalidierung clientseitig erfolgt.

Agent. Die kundenbetriebene, rein ausgehend verbindende Software. Sie ist Verwahrer des Origin-Schlüssels, prägt *Capabilities* und leitet Verkehr an den lokalen Origin weiter. Weil der Agent nur ausgehende Verbindungen aufbaut, benötigt der Kunde keine eingehende Firewall-Regel und keine öffentliche IP; die Erreichbarkeit entsteht allein am Edge.

Edge. Der betreiberbetriebene, öffentlich erreichbare Knoten. Er koordiniert das Rendezvous und leitet — als Rückfall — ausschließlich Chiffretext weiter. Er kann nicht entschlüsseln und routet über ein opakes Routing-Token. Er trägt weder SNI noch Hostnamen; ein Beobachter des Edge lernt lediglich, *dass* ein Tunnel existiert, nicht *wohin* er führt.

Client. Der Endpunkt, der einen Origin erreichen möchte. Er hält eine Capability und pinnt die Origin Identity. Der Client kann als Einmal-Tunnel (“single one-shot”) oder als lokaler Weiterleitungs-Port (*forward mode*) betrieben werden.

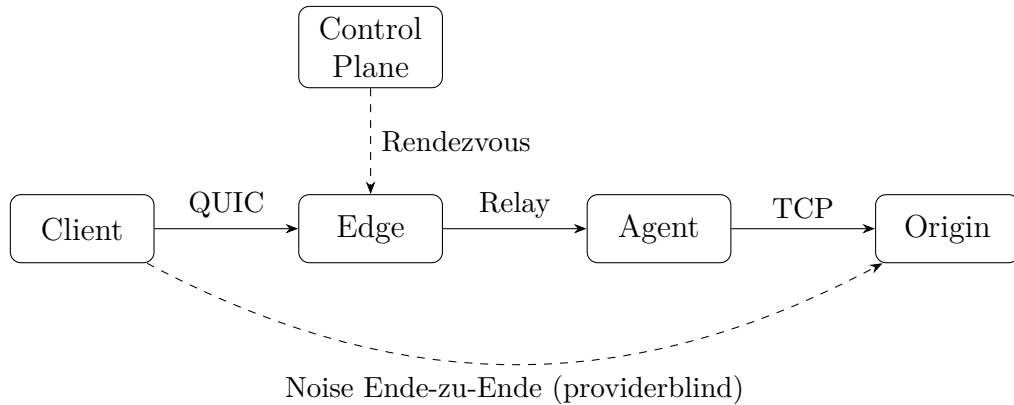


Abbildung 5.1: Komponenten und Datenpfad. Der Edge leitet nur Chiffretext weiter; die Noise-Sitzung ist Ende-zu-Ende zwischen Client und Origin und für den Edge undurchsichtig.

Control Plane. Eine dünne, self-hosting-fähige Steuerkomponente für Enrollment, Tunnel-Registry und Rendezvous-Endpunkt (ADR-0017). Sie hält weder Vertrauensmaterial noch Nutzlast. Über OIDC/Keycloak-verifizierte `/me/*`-Endpunkte verwaltet sie Konten und ein Kredit-Ledger und veröffentlicht die CA-Wurzel des Edge (Abschnitt 5.6).

Die Rollen sind bewusst so geschnitten, dass Vertrauen und Erreichbarkeit entkoppelt sind: Der Edge stellt Erreichbarkeit bereit, ohne Vertrauen zu besitzen; der Agent besitzt Vertrauen, ohne erreichbar zu sein. Der Client verbindet beide, indem er eine Capability vorlegt, die genau die für seine Sitzung nötige Autorität transportiert und nicht mehr.

5.2 Schlüsselflüsse

1. **Agent-Enrollment.** Ein einmaliges Join-Token bindet einen neu erzeugten Agent-Identitätsschlüssel an einen Mandanten; im Betrieb authentisiert sich der Agent über kurzlebigen mTLS. Die dafür nötige CA-Wurzel bezieht der Agent self-serve über die Control Plane (Abschnitt 5.6), sodass kein manuelles Kopieren von Vertrauensmaterial mehr erforderlich ist.
2. **Capability-Minting.** Der Agent erzeugt eine selbsttragende *Capability* — Routing-Token, Origin Identity und Edge-Adresse — die der Kunde außerhalb des Bandes an berechnete Clients verteilt (ADR-0014). Besitz genügt, um einen Origin zu erreichen und zu authentisieren. Weil die Capability den öffentlichen Schlüssel des Origin (Origin Identity) mitführt, ist der Client-seitige Vertrauensanker in ihr enthalten und muss nicht über eine CA-Hierarchie hergeleitet werden.

3. **Verbindungsaufbau.** Der Client legt seine Capability vor, durchläuft ein PoW- und ratenbegrenztes Rendezvous am Edge, wird über sein Routing-Token an den registrierten Agent geroutet und führt schließlich den Noise-Handshake zum Origin, gepinnt auf die Origin Identity. Der PoW-Nachweis (nach dem Hashcash-Prinzip [2]) verteuert Verbindungsversuche und ist damit die erste Verteidigungslinie gegen Sybil-/DoS-Fluten (ADR-0018).
4. **Revocation.** Widerruf erfolgt durch Rotation des Tokens und/oder des Origin-Schlüssels. Die Schlüsselrotation ist so entworfen, dass sie ohne Ausfallzeit erfolgt und bestehende Clients während eines definierten Fensters weiter bedient werden (Abschnitt 5.6).

5.3 Rollen-Dispatch am Edge

Der Prototyp bündelt die Edge-Logik in einer einzigen Routine, die pro eingehender Verbindung anhand eines ersten Rollen-Bytes verzweigt:

Rolle 'A' (Agent). Der Agent meldet auf einem Kontrollstrom sein Routing-Token an; der Edge trägt die Tunnel-Verbindung in die Registry ein und quittiert. Das Token wird damit routbar. Meldet sich ein zweiter Agent mit demselben Token an, ergänzt der Edge dessen Registrierung, statt die erste zu ersetzen — die Grundlage der Redundanz aus Abschnitt 5.4.

Rolle 'C' (Client). Der Client durchläuft das PoW-Rendezvous, präsentiert sein Routing-Token, woraufhin der Edge die passende Agent-Verbindung nachschlägt, einen Strom zu ihr öffnet und beide Ströme byteweise kopiert (Relay).

Dieses Dispatch bildet die Trennung von Registrierung und Nutzung auf einem gemeinsamen QUIC-Transport ab und hält den Edge zustandsarm: Er kennt nur die Abbildung Token → Agent-Verbindungen, nie den Klartext. Der Relay selbst ist rein bidirektionales Byte-Kopieren über die Noise-Chiffretexte; der Edge interpretiert den Inhalt weder noch kann er ihn interpretieren, da der Sitzungsschlüssel nur Client und Origin bekannt ist (ADR-0013).

5.4 Verfügbarkeit: Redundanz, Failover und Reconnect

Ein einzelner Agent ist ein Single Point of Failure für den von ihm bedienten Origin. Der Entwurf adressiert dies auf drei Ebenen, die zusammen die Blind-DoS-Resistenz und Verfügbarkeit aus ADR-0018 tragen.

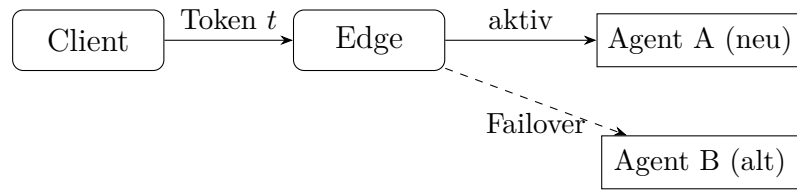


Abbildung 5.2: EdgeState-Registry für ein Routing-Token t . Der Edge routet newest-first auf Agent A; fällt dessen Verbindung aus, übernimmt Agent B. Beide Agenten bedienen denselben privaten Origin unter derselben Identität.

5.4.1 EdgeState-Registry und newest-first Failover

Statt eine Eins-zu-eins-Abbildung $\text{Token} \rightarrow \text{Agent}$ zu führen, hält der Edge für jedes Routing-Token eine *Liste* registrierter Agent-Verbindungen (konzeptionell $\text{Map} \langle \text{Routing-Token}, \text{Liste} \langle (\text{Agent-Id}, \text{Verbindung}) \rangle \rangle$). Mehrere Agenten teilen sich dieselbe Origin-Identität und dasselbe Token; jeder von ihnen kann denselben privaten Origin bedienen. Trifft eine Client-Anfrage ein, wählt der Edge die *neueste* Registrierung (newest-first) und leitet den Verkehr auf sie. Ist diese Verbindung nicht mehr nutzbar, rückt die nächste in der Liste nach. Abbildung 5.2 skizziert diese Registry.

5.4.2 Erkennung toter Agenten

Ein hart terminierter Agent sendet kein QUIC-CLOSE; ohne Gegenmaßnahme bliebe seine Registrierung als “Geisterroute” in der Liste. Der Edge löst dies transportnah: Der QUIC-Server aktiviert am Edge einen `keep_alive` von 3s und ein `max_idle_timeout` von 10s. Eine Verbindung, über die innerhalb des Idle-Fensters kein Verkehr und keine Keep-Alive-Antwort mehr fließt, wird vom Edge verworfen und ihre Registrierung aus der Token-Liste entfernt. Anschließend routet das newest-first-Verfahren aus Abschnitt 5.4.1 automatisch auf die überlebende Registrierung.

Dieses Verhalten wurde im Testbett verifiziert (Kapitel 7): Zwei Agenten teilen eine Origin-Identität, der Basisverbindungsaufbau erfolgt `via=quic`; nach dem gezielten Abschuss des bedienenden Agenten schlägt der nächste Client-Round-Trip zunächst fehl, und nach Ablauf des Idle-Fensters gelingt er erneut `via=quic` über den verbliebenen Agenten (“REDUNDANCY OK”). Das Fenster ist damit die Obergrenze der Ausfalldauer bei stillem Agent-Verlust.

5.4.3 Reconnect-on-drop

Symmetrisch zur edge-seitigen Eviktion behandelt der Agent den Verlust seiner Kontrollverbindung (#5): Bricht die QUIC-Verbindung zum Edge ab, versucht

der Agent den erneuten Aufbau mit exponentiellem Backoff und einer Obergrenze (*capped exponential backoff*). So kehrt ein nur vorübergehend gestörter Agent selbsttätig in die Registry zurück, ohne den Edge mit Verbindungsstürmen zu belasten. Redundanz (mehrere Agenten) und Reconnect (Selbstheilung eines Agenten) ergänzen sich: Erstere überbrückt den harten Ausfall sofort, letztere stellt die volle Kapazität ohne manuellen Eingriff wieder her.

5.5 Beobachtbarkeit

Weil der Edge providerblind ist, darf Beobachtbarkeit ausschließlich auf strukturellen Metadaten beruhen — niemals auf Nutzlast (ADR-0016). Der Edge exponiert daher einen Prometheus-kompatiblen Endpunkt `/metrics` mit Zählern und Gauges, die genau diese unbedenklichen Größen abbilden:

ct_edge_active_tunnels (Gauge) aktuell gekoppelte Client-Agent-Relays.

ct_edge_active_agents (Gauge) aktuell registrierte Agent-Verbindungen.

ct_edge_registrations_total (Counter) kumulierte Agent-Registrierungen.

ct_edge_relays_total und **ct_edge_relay_bytes_total** (Counter) Anzahl bzw. Bytevolumen der Relays.

ct_edge_failovers_total (Counter) Anzahl der Failover-Ereignisse.

Keiner dieser Werte offenbart, *wer* mit *wem* kommuniziert oder *was* übertragen wird; sie zählen lediglich Ereignisse und Volumina. Ein realer Scrape des laufenden Self-Hosts belegte etwa `ct_edge_registrations_total = 7`, `ct_edge_relays_total = 2` und `ct_edge_relay_bytes_total = 8369`, bei zum Scrape-Zeitpunkt keinem registrierten Agenten (Gauges = 0); die Zahlen sind in Kapitel 7 eingeordnet.

Ergänzend bietet die Control Plane einen `/status`-Endpunkt und eine self-contained Operator-Landingpage (#4) unter `/`: eine eigenständige HTML-Seite ohne externe Abhängigkeiten, die ihre Kennzahlen aus `/status` bezieht. Wichtig für die Blindheit: Die Statusseite meldet die *live* Tunnelzahl, die aus dem Edge-/metrics-Gauge `ct_edge_active_tunnels` stammt, nicht aus der Rendezvous-Registry. Der `/status`-Korpus umfasst zusätzlich betriebliche Aggregate wie Bereitschaftsflag, registrierte Agenten, Konten und bestätigte Zahlungen — allesamt zählende, nicht inhaltliche Größen.

5.6 Vertrauensverteilung und Schlüsselrotation

5.6.1 Self-serve CA-Root-Verteilung

Damit sich Agenten per mTLS gegenüber dem Edge authentisieren können, benötigen sie dessen CA-Wurzel. Anstatt dieses Vertrauensmaterial außerhalb des Bandes zu kopieren, veröffentlicht die Control Plane es self-serve unter `/pki/ca` (#11). Agenten und Clients beziehen die CA-Wurzel dort und pinnen sie. Das reduziert Fehlkonfiguration und macht das Enrollment reproduzierbar, ohne die Zero-Knowledge-Grenze zu verletzen: Die CA-Wurzel ist öffentliches Vertrauensmaterial, kein Geheimnis; der Edge bleibt ohne Kenntnis von Origin-Schlüsseln oder Nutzlast.

5.6.2 Zero-Downtime-Schlüsselrotation

Die Rotation eines Origin-Schlüssels muss Bestandsclients nicht abwerfen. Der Entwurf trennt dazu *Routing* von *Identität*: Das Routing-Token bleibt bei der Rotation erhalten, während der Agent eine *neue* Origin-Identität einführt und den alten Schlüssel für ein definiertes Fenster weiter vorhält (*Retired-Key-Fenster*, verwaltet über ein Schlüsselverzeichnis am Agenten). Während dieses Fensters gilt:

- Weil das Token unverändert bleibt, routen *alte* Clients weiterhin ohne Neuverteilung ihrer Capability.
- Der Agent bedient sowohl den *alten* (retired) als auch den *neuen* Schlüssel: Ein alter Client vollzieht den Noise-Handshake gegen den retired key, ein neuer Client gegen die frische Identität.

Im Testbett schlossen sowohl die *alte* als auch die *neue* Capability nach der Rotation je einen vollständigen Tunnel-Round-Trip ab (“ROTATION OK”). Nach Ablauf des Fensters entfällt der retired key, und nur die neue Identität bleibt gültig — ein expliziter, kundengesteuerter Widerruf ohne Auto-Ablauf (ADR-0014). Rotation ist damit zugleich der Revocation-Mechanismus aus Abschnitt 5.2.

5.7 Transport- und Applikationsflexibilität

5.7.1 QUIC-Primärtransport mit TLS-über-TCP-Rückfall

Der Datenpfad nutzt QUIC (quinn) als Primärtransport (ADR-0004). Ist UDP blockiert — etwa in restriktiven Netzen —, greift ein Rückfall auf TLS-über-TCP an Port 443, der denselben Tunnel trägt. Der Client kann den Rückfall erzwingen; im Testbett lieferte der Standardpfad “SMOKE OK via=quic”, der

erzwungene Rückfall “SMOKE OK via=tcp” — beide transportieren denselben Ende-zu-Ende-Tunnel. Die Transportwahl ist für die Sicherheit unerheblich, da die Noise-Sitzung oberhalb des Transports liegt.

5.7.2 Cross-Transport-Relay

Da der Edge nur Chiffretext koppelt, müssen Client und Agent nicht denselben Transport verwenden. Ein QUIC-Client kann einen Agenten erreichen, der im TCP-Rückfall registriert ist (#13); der Edge überbrückt die beiden Transporte, indem er die Ströme auf Byteebene verbindet. Damit ist die Transportwahl eine lokale Eigenschaft jedes Endpunkts und keine Eigenschaft des Tunnels als Ganzem.

5.7.3 TLS-at-origin und Client-Forward-Modus

Der Tunnel ist inhaltsneutral und trägt daher beliebigen TCP-Verkehr, einschließlich einer bereits durch TLS geschützten Verbindung. Im *HTTPS-at-origin*-Szenario (#22) terminiert ein realer HTTPS-Dienst das TLS am Origin selbst; der Edge relayt nur den TLS-über-Noise-Chiffretext und sieht weder das innere TLS noch dessen Klartext. Die Zertifikatsvalidierung geschieht *clientseitig*: Ein Aufruf mit gepinnter CA validiert das Origin-Zertifikat und erhält eine reguläre HTTP-200-Antwort durch den Tunnel. Der *Client-Forward-Modus* macht dies allgemein nutzbar — er öffnet einen lokalen TCP-Port, über den eine beliebige TCP-/TLS-Anwendung den Tunnel mitbenutzt, ohne den Tunnel zu “kennen”. Das demonstriert die providerblinde Zusicherung besonders deutlich: Selbst der TLS-Handshake der Anwendung bleibt für den Edge undurchsichtig, weil er innerhalb der Noise-Sitzung liegt.

5.8 Ablauf des Verbindungsaufbaus

Die vorstehenden Komponenten und Flüsse greifen beim eigentlichen Verbindungsaufbau in einer festen Reihenfolge ineinander. Abbildung 5.3 stellt diesen als zeitlichen Ablauf über die vier Datenpfad-Rollen dar; die Zeit läuft nach unten, die gestrichelten Linien sind die “Lebenslinien” der Beteiligten.

Voraussetzung des Ablaufs ist, dass sich mindestens ein Agent bereits unter Rolle ‘A’ beim Edge angemeldet und damit sein Routing-Token routbar gemacht hat. Wünscht ein Client eine Verbindung, meldet er sich unter Rolle ‘C’; der Edge antwortet mit einer PoW-Challenge, die der Client löst und zusammen mit seinem Routing-Token vorlegt. Erst nach diesem ratenbegrenzten, nach dem Hashcash-Prinzip [2] verteuerten Rendezvous schlägt der Edge die zum Token passende Agent-Registrierung nach, öffnet einen Strom zu ihr und koppelt die beiden Ströme byteweise. Bis zu diesem Punkt hat der Edge nur

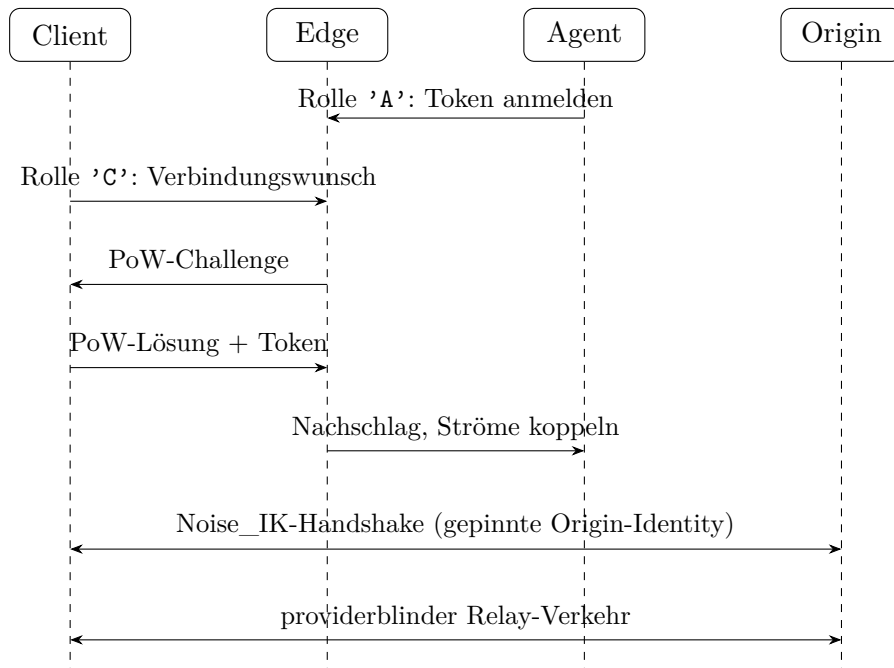


Abbildung 5.3: Zeitlicher Ablauf des Verbindungsaufbaus. Zuerst meldet der Agent unter Rolle 'A' sein Routing-Token an; anschließend durchläuft der Client (Rolle 'C') das PoW-Rendezvous und wird mit dem Agenten gekoppelt. Die beiden untersten Pfeile sind logisch Ende-zu-Ende zwischen Client und Origin — der Edge kreuzt sie lediglich als providerblinder Relay und sieht ausschließlich Chiffretext.

strukturelle Metadaten gesehen: das opake Token, den PoW-Nachweis und die schiere Existenz einer Kopplung.

Auf der so hergestellten Byte-Brücke vollzieht der Client anschließend den `Noise_IK`-Handshake direkt mit dem Origin, gepinnt auf die in der Capability mitgeführte Origin Identity; der Edge relayt dabei nur Chiffretext und kann den Handshake weder lesen noch verändern (ADR-0013). Ist die Sitzung etabliert, fließt der eigentliche Nutzverkehr als *providerblinder* Relay: Client und Origin teilen den Sitzungsschlüssel, während Edge und Agent ausschließlich verschlüsselte Bytes weiterreichen. Damit realisiert der Ablauf die Zero-Knowledge-Grenze aus ADR-0002 über die gesamte Lebensdauer der Verbindung, nicht nur während des Aufbaus.

5.9 Ablauf des Failovers

Der in Abschnitt 5.4 beschriebene Verfügbarkeitsentwurf wird beim Ausfall des bedienenden Agenten in einer festen Sequenz wirksam. Sie verbindet die redundante Registry (Abschnitt 5.4.1) mit der transportnahen Erkennung toter Agenten (Abschnitt 5.4.2) zu einem selbstheilenden Failover:

1. Zwei Agenten teilen dieselbe Origin-Identität und dasselbe Routing-Token und melden sich beide unter Rolle 'A' an; der Edge führt sie als zwei Einträge in der Token-Liste.
2. Der Edge routet newest-first auf den zuletzt registrierten, bedienenden Agenten; der Basis-Round-Trip des Clients gelingt `via=quic`.
3. Der bedienende Agent stirbt hart und sendet kein `QUIC-CLOSE`; die tote Verbindung verbleibt zunächst als "Geisterroute" in der Liste, und der unmittelbar folgende Client-Round-Trip schlägt fehl.
4. Nach Ablauf des Idle-Fensters (`keep_alive 3s, max_idle_timeout 10s`) verwirft der Edge die stille Verbindung und entfernt ihre Registrierung aus der Token-Liste.
5. Das newest-first-Routing wählt nun den überlebenden Agenten; der nächste Client-Round-Trip gelingt erneut `via=quic` ("REDUNDANCY OK").

Das Idle-Fenster ist damit die Obergrenze der Ausfalldauer bei stillem Agent-Verlust: Solange mindestens ein Agent dieselbe Origin-Identität bedient, überbrückt die Redundanz den harten Ausfall automatisch, ohne Neuverteilung von Capabilities und ohne manuellen Eingriff. Ergänzend führt der Reconnect-on-drop-Mechanismus (Abschnitt 5.4.3) einen nur vorübergehend gestörten Agenten selbsttätig in die Registry zurück, sodass die volle Redundanz nach der Störung wiederhergestellt ist. Zusammen tragen diese Schritte die Verfügbarkeits- und Blind-DoS-Härtung aus ADR-0018, ohne die providerblinde Zusicherung aufzuweichen.

5.10 Entwurfsentscheidungen

Die Architektur folgt einigen bewusst getroffenen Entscheidungen, die in Architecture Decision Records festgehalten sind:

- ein providerblinder, Ende-zu-Ende-verschlüsselter Datenpfad (ADR-0001) mit klar definierter Zero-Knowledge-Grenze (ADR-0002);
- agentgehaltene Zertifikate und out-of-band verteilte Capabilities (ADR-0003, ADR-0014);
- QUIC [9, 17] als Transport mit TLS-über-TCP-Rückfall (ADR-0004) und responsiver Terminierung (ADR-0008);
- die Noise-basierte, PKI-freie Mesh-Authentisierung (ADR-0013, [12]) mit Mesh-Plane-First-Zuschnitt (ADR-0010);
- agentseitige Beobachtbarkeit auf reinen Metadaten (ADR-0016);
- ein einregionaler Edge um eine erststufige Tunnel-Registry (ADR-0006) mit Verfügbarkeits- und Blind-DoS-Härtung (ADR-0018);

5 Architektur

- sowie eine dünne, self-hosting-fähige Control Plane, die weder Schlüssel noch Nutzlast sieht (ADR-0017).

Diese Entscheidungen greifen ineinander und lassen sich entlang der beiden tragenden Ziele — *Providerblindheit* und *Verfügbarkeit* — gruppieren. Die Providerblindheit ruht auf ADR-0001 und ADR-0002: Der Datenpfad ist Ende-zu-Ende verschlüsselt, und die Zero-Knowledge-Grenze legt formal fest, welche Größen der Edge überhaupt sehen darf (opakes Token, PoW-Ereignisse, strukturelle Zähler) und welche nicht (Klartext, Schlüssel, Zielidentität). Die Transportentscheidung ADR-0004 ist dieser Grenze bewusst untergeordnet: Ob der Tunnel über QUIC oder den TLS-über-TCP-Rückfall läuft, ist sicherheitsneutral, weil die *Noise_IK*-Sitzung oberhalb des Transports liegt; der Cross-Transport-Relay und der TLS-at-origin-Fall in Abschnitt 5.7 sind unmittelbare Folgen davon. Dass die Control Plane dünn und self-hosting-fähig bleibt (ADR-0017), erweitert dieselbe Blindheit auf die Steuerebene: Sie vermittelt Enrollment und Rendezvous, hält aber weder Schlüsselmaterial noch Nutzlast, sodass auch ein Betreiber der Steuerkomponente keine Vertraulichkeit brechen kann.

Der Verfügbarkeitsentwurf ordnet sich derselben Leitlinie unter, statt sie aufzuweichen. Der einregionale Edge um eine erststufige Tunnel-Registry (ADR-0006) hält den Zustand am Edge bewusst klein — nur die Abbildung Token → Agent-Verbindungen — und macht so die newest-first-Redundanz und die transportnahe Erkennung toter Agenten (Abschnitt 5.4) überhaupt erst zustandsarm umsetzbar. ADR-0018 ergänzt diese Basis um die Verfügbarkeits- und Blind-DoS-Härtung: Das PoW-Rendezvous ist die erste Verteidigungslinie gegen Fluten, ohne dass der Edge Identitäten oder KYC benötigt und damit seine Blindheit preisgäbe. Der bewusst schmale v1-Zuschnitt schließlich — Mesh-Plane-First mit zurückgestellter Browser Plane (ADR-0010) — hält den Prototyp auf genau den Eigenschaften fokussiert, die die providerblinde Zusicherung belegen. Bewusst zurückgestellt ist die *Browser Plane* (öffentlicher Hostname mit Let's-Encrypt-Zertifikat über SNI); v1 beweist stattdessen, dass TLS am Origin terminiert (ADR-0010). Ebenso ist der Multi-Region-/Mesh-P2P-Ausbau (ADR-0015) nur teilweise realisiert (Direktpfad-Kandidat mit Relay-Rückfall). Kapitel 6 zeigt, wie diese Entscheidungen in den Rust-Crates umgesetzt sind.

6 Implementierung

Dieses Kapitel dokumentiert den Prototyp: den Aufbau des Rust-Workspace (Abschnitt 6.1), die kryptographischen Bausteine (Abschnitt 6.2), das Rollen-Dispatch am Edge (Abschnitt 6.3), Agent und Client (Abschnitt 6.4) sowie die darüber hinaus ausgelieferten Betriebsfunktionen: Agent-Redundanz mit Failover (Abschnitt 6.5), Observability am Edge (Abschnitt 6.6), die selbstbediente Verteilung des CA-Roots (Abschnitt 6.7), die unterbrechungsfreie Schlüsselrotation (Abschnitt 6.8), das Wiederverbinden des Agenten (Abschnitt 6.9), das transportübergreifende Relay (Abschnitt 6.10) und den HTTPS-Betrieb am Origin (Abschnitt 6.11). Abschnitt 6.12 fasst die Qualitätssicherung zusammen.

6.1 Aufbau des Workspace

Der Prototyp ist ein Rust-Cargo-Workspace aus fünf Crates (Tabelle 6.1). Der Datenpfad ist bewusst in Rust umgesetzt – speichersicher ohne Garbage-Collector, mit ausgereiften Bibliotheken für die benötigten Bausteine: `quinn` für QUIC, `rustls` (mit dem `ring`-Provider) für die Transportsicherung, `snow` für den Noise-Handshake, `ed25519-dalek` für Signaturen, `sha2` für den Proof-of-Work, `rcgen` für selbstsignierte Zertifikate und `tokio` als asynchrone Laufzeit.

Die Schnittebene zwischen den Crates folgt der Zonentrennung aus Kapitel 5: `ct-common` enthält ausschließlich reine, zustandslose Bausteine ohne Netzwerk- oder Dateisystemzugriff und ist daher zu 100 % mit Einheitstests abdeckbar; die drei Rollen-Binaries (`ct-edge`, `ct-agent`, `ct-client`) halten je einen schmalen `main`-Einstiegspunkt, dessen eigentliche Logik in einer Bibliotheksschicht liegt, damit sie ohne echten Prozessstart integrativ getestet werden kann. `ct-control-plane` ist die einzige Komponente mit persistentem Zustand (Konto- und Kreditregister, Enrollment-Registry) und hält gemäß ADR-0017 weder Schlüssel noch Nutzlast.

6.2 Kryptographische Bausteine (`ct-common`)

`ct-common` bündelt die geteilten Typen. Eine `Capability` (Routing-Token, Origin Identity, Edge-Adresse) besitzt eine handgeschriebene binäre Kodierung

Tabelle 6.1: Crates des Workspace und ihre Rolle.

Crate	Rolle
<code>ct-common</code>	Gemeinsame Typen: Capability, Credential, Noise, PoW, Ratenbegrenzung
<code>ct-edge</code> <code>ct-agent</code>	Edge-Daemon: Registry, Rollen-Dispatch, Relay Agent-Daemon: Tunnel-Registrierung, Weiterleitung zum Origin
<code>ct-client</code>	Client-Werkzeug: Rendezvous, Tunnel, Bench-Modus
<code>ct-control-plane</code>	Dünne Steuerkomponente: Enrollment, Registry, Ausstellung

mit definierter Fehlerbehandlung (`MalformedCapability`) statt einer generischen Serialisierung, um das Wire-Format kontrolliert zu halten. Der Proof-of-Work ist bewusst asymmetrisch: Das Prüfen kostet einen einzigen Hash, das Lösen wächst exponentiell mit der Schwierigkeit (Listing 6.1). Hinzu kommen der Noise-Handshake (Client- und Origin-Seite über `snow`), die signierten Credentials (`ed25519-dalek`) und eine Fenster-basierte Ratenbegrenzung pro Token.

Der Noise-Handshake nutzt das Muster `Noise_IK` mit der Suite X25519 / ChaCha20-Poly1305 / BLAKE2s (ADR-0013). IK passt zum Bedrohungsmodell (Kapitel 5): Der Client kennt die statische öffentliche Origin-Identität aus seiner Capability vorab (`I = known`), während er selbst dem Origin gegenüber erst im Handshake identifiziert wird. Das Routing-Token ist ein opakes Byte-Feld ohne Bezug zu SNI oder Hostname; der Edge löst darüber nur die Zielverbindung auf und kann aus dem Token weder den Origin noch den Kunden ableiten. Die Ratenbegrenzung führt pro Token ein gleitendes Zeitfenster, so dass sich der Proof-of-Work-Aufwand nicht durch Wiederholung derselben gelösten Challenge umgehen lässt — Challenge-Nonce und Fenster zusammen bilden die primäre Sybil- und DoS-Schranke (ADR-0018) in Abwesenheit einer KYC-Prüfung.

Der Proof-of-Work folgt der Hashcash-Konstruktion [2]: Ein Löser sucht eine Zahl, deren Hash mit hinreichend vielen führenden Nullbits beginnt; die Prüfung ist ein einzelner Hashvergleich. Damit lässt sich die Zutrittskosten eines Rendezvous über einen einzelnen Parameter (`EDGE_POW_DIFFICULTY`) skalieren, ohne dass die Verifikation am Edge teurer wird.

```
// Verify: ein einziger Hash.
pub fn verify(challenge: &Challenge, solution: u64) -> bool {
    leading_zero_bits(&hash(&challenge.nonce, solution))
        >= challenge.difficulty as u32
}

// Solve: Brute force, Kosten ~2^difficulty.
pub fn solve(challenge: &Challenge) -> u64 {
```

```

let mut solution = 0u64;
loop {
  if verify(challenge, solution) { return solution; }
  solution += 1;
}

```

Listing 6.1: Asymmetrischer Proof-of-Work: Prüfen ist billig, Lösen teuer (ct-common::pow).

6.3 Rollen-Dispatch am Edge (ct-edge)

Der Kern des Edge ist die Routine `serve_connection` (Listing 6.2): Sie liest ein erstes Rollen-Byte und verzweigt. Ein Agent ('A') meldet sein Token an und wird in die Registry (`EdgeState`) eingetragen; ein Client ('C') erhält die PoW-Challenge, sein Request wird geprüft, das Token zur Agent-Verbindung aufgelöst und beide QUIC-Ströme werden byteweise gekoppelt. Der Edge hält damit nur die Abbildung Token → Verbindung und sieht nie den Klartext: Zwischen Client und Origin liegt der Noise_IK-Kanal, dessen Chiffretext der Edge lediglich weiterreicht (ADR-0001, ADR-0002).

```

match role[0] {
  b'A' => {
    // Agent: Tunnel registrieren
    recv.read_exact(&mut token).await?;
    state.register(RoutingToken(token), conn.clone());
    send.write_all(b"OK").await?;
  }
  b'C' => {
    // Client: PoW-Rendezvous, dann Relay
    send.write_all(&challenge_bytes).await?;
    recv.read_exact(&mut req).await?;
    let token = check_request(challenge, &req)?; // PoW pruefen
    let agent = state.route(&token).ok_or("no agent")?;
    let (a_send, a_recv) = agent.open_bi().await?;
    relay_quic(send, recv, a_send, a_recv).await?; // koppeln
  }
  other => return Err(format!("unknown role: {other}").into()),
}

```

Listing 6.2: Rollen-Dispatch am Edge (gekürzt, ct-edge::serve).

6.4 Agent und Client

Der Agent (ct-agent) wählt ausgehend zum Edge, registriert seinen Tunnel ('A') und überbrückt anschließend jeden vom Edge weitergeleiteten QUIC-Strom mittels `copy_bidirectional` auf eine TCP-Verbindung zum lokalen Origin. Der Client (ct-client) wählt den Edge, durchläuft das PoW-Rendezvous

(`'C'`), tunnelt seine Nutzlast und verifiziert die Antwort; ein zusätzlicher Bench-Modus misst die Roundtrip-Latenz über mehrere Iterationen (`CT_CLIENT_ITERATIONS`) und gibt eine beschriftete CSV-Zeile mit Mittelwert, Minimum, Maximum, den Perzentilen p50/p95/p99, Standardabweichung und 95 %-Konfidenzintervall aus. Diese Zeile ist die Datenquelle der Evaluation (Kapitel 7). Alle drei Binaries beziehen ihre Konfiguration aus Umgebungsvariablen und koordinieren sich im Testbett über ein geteiltes Volume (Edge-Zertifikat, Capability).

Der primäre Transport ist QUIC über `quinn` (ADR-0004). Ist UDP im Netz blockiert, fällt der Client auf eine TLS-über-TCP-Verbindung nach Port 443 zurück; dieselbe Tunnel-Semantik wird dann über den TCP-Pfad getragen. Der Fallback lässt sich zur Reproduktion erzwingen (`CT_CLIENT_FORCE_TCP=1`); der Cross-Host-Rauchtest bestätigt für beide Pfade einen erfolgreichen Roundtrip (`"SMOKE OK via=quic"` bzw. `"via=tcp"`).

6.5 Agent-Redundanz und Failover (ct-edge)

Damit ein Origin mehr als einen anmeldenden Agenten tragen kann, ist die Registry nicht als 1:1-Abbildung, sondern als `HashMap<RoutingToken, Vec<(id, Connection)>` ausgelegt. Mehrere Agenten teilen sich also dieselbe Origin-Identität und dasselbe Routing-Token und tragen sich als getrennte Einträge derselben Liste ein. Beim Auflösen eines Client-Requests wählt `route` den zuletzt registrierten Eintrag zuerst (*newest-first*) und probiert die weiteren als Rückfallebene durch.

Fällt der bedienende Agent hart aus (Prozessabbruch ohne `QUIC-CLOSE`), kann der Edge dies nicht sofort erkennen, weil keine ordentliche Verbindungstrennung eintrifft. Deshalb setzt der Edge-Server für jede eingehende Verbindung `keep_alive` auf 3 s und `max_idle_timeout` auf 10 s: Bleibt eine Registrierung länger als das Leerlauffenster stumm, verwirft QUIC die Verbindung, und der Eintrag wird aus der Liste entfernt (Eviction). Der nächste Client-Roundtrip löst dann auf die überlebende Registrierung auf. Der Rauchtest `redundancy-smoke.sh` belegt dies: Zwei Agenten teilen eine Origin-Identität, der bedienende Agent wird getötet, und nach Ablauf des Erkennungsfensters gelingt der Roundtrip erneut `"via=quic"` über den zweiten Agenten (`"REDUNDANCY OK"`).

6.6 Observability am Edge (ct-edge)

Der Edge exponiert einen `/metrics`-Endpunkt im Prometheus-Textformat (ADR-0016). Gemessen werden zwei Momentanwerte (Gauges) — die Zahl der aktiven Tunnel (`ct_edge_active_tunnels`) und der aktuell registrierten Agenten (`ct_edge_active_agents`) — sowie vier kumulative Zähler (Counter):

abgeschlossene Registrierungen (`ct_edge_registrations_total`), aufgebaute Relays (`ct_edge_relays_total`), das über Relays getragene Byte-Volumen (`ct_edge_relay_bytes_total`) und die Zahl der Failover-Ereignisse (`ct_edge_failovers_total`). Alle Werte sind bewusst *strukturell*: Sie beschreiben Existenz, grobe Zeit und Volumen von Tunneln, aber niemals deren Inhalt — der Edge bleibt damit auch in der Observability zonentreu (ADR-0002).

Eine reale Momentaufnahme des laufenden Selbst-Hosts zeigt das Zusammenspiel: zum Scrape-Zeitpunkt war kein Agent angemeldet (`ct_edge_active_tunnels` 0, `ct_edge_active_agents` 0), über die Laufzeit wurden aber 7 Registrierungen und 2 Relays mit zusammen 8369 Byte Nutzvolumen gezählt, bei 0 Failovern. Die dünne Steuerkomponente greift diese Zahlen auf: Ihre Statusseite und der `/status`-Endpoint melden die *live* vom Edge gescrapte Tunnelzahl (nicht den Inhalt der Rendezvous-Registry) und ergänzen betriebliche Kennzahlen der Steuerebene — in derselben Aufnahme 45 Agenten insgesamt, 4 Konten und 2 bestätigte Zahlungen. Eine selbsttragende HTML-Landeseite (Feature #4) rendert diese Zählwerte ohne externe Abhängigkeiten für den Betreiber.

6.7 Selbstbediente Verteilung des CA-Roots (`ct-control-plane`)

Damit Agenten und Clients dem Edge-Transport vertrauen können, benötigen sie dessen CA-Root-Zertifikat. Statt es außerhalb des Systems zu kopieren, veröffentlicht die Steuerkomponente das Edge-CA-Root unter `/pki/ca`. Agenten und Clients holen es beim Start von dort. Der Bezug betrifft ausschließlich das öffentliche Transport-Root der Steuerebene; die end-to-end-relevanten Origin-Schlüssel bleiben davon unberührt und werden gemäß ADR-0014 als Out-of-Band-Capabilities in der Verantwortung des Kunden verteilt. Die Steuerebene hält damit weiterhin keinerlei geheimes Schlüsselmaterial des Datenpfads.

6.8 Unterbrechungsfreie Schlüsselrotation (`ct-agent`)

Die Rotation der Origin-Identität ist so entworfen, dass laufende Clients nicht brechen. Der Kniff ist die Trennung von *Routing* und *Identität*: Bei einer Rotation behält der Origin sein Routing-Token, erhält aber eine neue Origin-Identität; die Capability wird über `mint_capability_with_token` mit unverändertem Token neu ausgestellt. Weil das Token gleich bleibt, routen bereits verteilte (alte) Clients weiterhin auf denselben Origin.

Für ein Zeitfenster bedient der Agent parallel den zurückgezogenen Schlüssel: Alte Schlüssel wandern in ein Verzeichnis retirierter Schlüssel (`CT_AGENT_ORIGIN_KEY_DIR`),

gegen die der Agent den `Noise_IK`-Handshake während der Übergangszeit weiterhin abschließt. So handshaken alte Clients noch die retirierte Identität, während neue Clients die neue Identität nutzen. Der Rauchtest `rotation-smoke.sh` weist beides nach: Nach der Rotation mit demselben Token schließen *sowohl* die alte *als auch* die neue Capability einen vollständigen Tunnel-Roundtrip ab (“ROTATION OK”). Die Rotation ist explizit und ohne automatischen Ablauf — der Zeitpunkt der endgültigen Ausmusterung bleibt eine bewusste Betreiberentscheidung.

6.9 Wiederverbinden des Agenten (`ct-agent`)

Ein Agent muss transiente Netzstörungen und Edge-Neustarts überstehen, ohne seinen Tunnel dauerhaft zu verlieren. Bricht die QUIC-Verbindung zum Edge ab, versucht der Agent, sie erneut aufzubauen und sich neu zu registrieren. Die Wiederverbindungsversuche folgen einem exponentiellen Backoff mit oberer Schranke: Die Wartezeit verdoppelt sich nach jedem Fehlversuch, bis eine Obergrenze erreicht ist, ab der in konstantem Abstand weiter versucht wird. Das begrenzt die Last auf einen gerade neu startenden Edge und vermeidet zugleich, dass ein länger gestörter Agent unnötig lange offline bleibt. In Verbindung mit der Eviction aus Abschnitt 6.5 ergibt sich ein selbstheilendes Verhalten: Ein zwischenzeitlich ausgefallener Agent trägt sich nach Wiederkehr erneut in die Registry ein.

6.10 Transportübergreifendes Relay (`ct-edge`)

Client und Agent müssen nicht denselben Transport benutzen. Der Edge koppelt die beiden Seiten eines Tunnels unabhängig davon, ob sie über QUIC oder über den TLS-über-TCP-Fallback (Abschnitt 6.4) angebunden sind. So kann ein QUIC-Client einen Agenten erreichen, der nur über den TCP-Fallback am Edge hängt (Feature #13), und umgekehrt. Da der Edge lediglich Chiffretext zwischen den beiden Strömen weiterreicht und den `Noise_IK`-Kanal nicht terminiert, ist die Transportwahl der einen Seite für die andere transparent; die Ende-zu-Ende-Sicherung bleibt in jedem Fall zwischen Client und Origin bestehen.

6.11 HTTPS am Origin und Client-Forward-Modus (`ct-client`)

Ein zentrales Ziel der Zonentrennung ist, dass TLS am Origin terminiert und der Edge auch dann nur Chiffretext sieht, wenn der Origin selbst HTTPS spricht. Der Prototyp weist dies end-to-end nach: Eine reale, selbstsignierte

HTTPS-Seite (SAN 127.0.0.1) auf Loopback wird durch den Tunnel erreicht, und `curl -cacert` liefert HTTP 200 mit *client-seitig* validiertem Zertifikat des Origins (Rauchtest `https-demo.sh`). Die TLS-Sitzung läuft dabei zwischen Browser bzw. Client und Origin; der Edge relayt nur das in Noise gekapselte TLS-Chiffretext. Der gefrorene Integrationstest `https_website_through_the_tunnel_with_c` sichert dieses Verhalten dauerhaft ab.

Damit beliebige TCP/TLS-Anwendungen den Tunnel nutzen können, kennt der Client einen Weiterleitungsmodus (`CT_CLIENT_MODE=forward`): Er öffnet einen lokalen Port, über den jede Anwendung ihren Verkehr in den Tunnel schickt, anstatt selbst die Nutzlast zu formen (Listing 6.3). Erst dieser Modus macht aus dem Tunnel ein universelles Transport-Werkzeug statt eines demonstrativen Roundtrip-Klienten.

```
let listener = TcpListener::bind(local_addr).await?;
loop {
    let (mut app, _) = listener.accept().await?; // lokale App
    let mut tunnel = open_tunnel(&edge, &cap).await?; // Noise ueber QUIC
    copy_bidirectional(&mut app, &mut tunnel).await?; // koppeln
}
```

Listing 6.3: Client-Forward-Modus: lokaler Port in den Tunnel (gekürzt, `ct-client`).

Damit ist die im Bedrohungsmodell geforderte Eigenschaft belegt, dass TLS am Origin terminiert; die weitergehende *Browser Plane* mit öffentlichem Hostnamen und Let's-Encrypt-Zertifikaten über SNI ist bewusst auf die Zeit nach v1 verschoben (ADR-0010).

6.12 Qualitätssicherung

Die Umsetzung ist testgetrieben entstanden. Der Workspace umfasst 266 automatisierte Tests, die hermetisch bestehen — verteilt auf `ct-common` (40), `ct-agent` (65), `ct-client` (30), `ct-edge` (52) und `ct-control-plane` (79). Sie reichen von den reinen Kodierungs- und PoW-Funktionen bis zu Integrationstests, die einen vollständigen Client → Edge → Agent-Roundtrip über echtes QUIC ausführen. Die auf die Bibliotheksschicht bezogene Zeilenabdeckung (ohne die dünnen Einstiegspunkte, gemessen mit `cargo-llvm-cov`) beträgt 95,57 % der Zeilen, 94,44 % der Funktionen und 93,74 % der Regionen; das Skript `scripts/coverage.sh` erzwingt eine untere Schranke von 95 % Zeilenabdeckung.

Als Regressionsschranke dient ein hermetisches Docker-Gate: Bau und Test des gesamten Workspace laufen in einem `rust:1-slim`-Container mit `RUSTFLAGS=-D warnings` und schlagen bei der ersten Warnung fehl; der Prototyp durchläuft dieses Gate mit 0 Warnungen. Zusammen mit den in den vorangehenden Abschnitten genannten Rauchtests (`e2e-smoke.sh`, `redundancy-smoke.sh`,

rotation-smoke.sh, https-demo.sh) ergibt sich eine reproduzierbare Qualitätsbasis, auf der die Evaluation in Kapitel 7 aufsetzt.

6.13 Ausgewählte Implementierungsdetails

Die vorangehenden Abschnitte beschreiben die Mechanismen auf Verhalten-sebene. Dieser Abschnitt vertieft drei Bausteine bis auf die Datenstruktur- und Wire-Ebene, weil sich an ihnen die Zonentrennung konkret ablesen lässt: die Registry am Edge (Abschnitt 6.13.1), das gerenderte `/metrics`-Dokument (Abschnitt 6.13.2) und die Kopplung im Client-Forward-Modus (Abschnitt 6.13.3).

6.13.1 Die EdgeState-Registry: Typ, Failover, Eviction

Die zentrale Datenstruktur des Edge ist eine einzige Abbildung vom opaken Routing-Token auf eine Liste registrierter Agent-Verbindungen (Listing 6.4). Der Werttyp ist bewusst ein `Vec` aus Paaren (`id`, `Connection`) und keine skalare Verbindung: Dadurch können sich mehrere Agenten unter demselben Token eintragen (Grundlage der Redundanz aus Abschnitt 6.5), und die `id` identifiziert einen konkreten Eintrag eindeutig für dessen spätere Entfernung. Die Abbildung enthält ausschließlich das Token und einen Verbindungshandle — keinen Origin-Namen, keinen Schlüssel, keine Nutzlast; sie ist die vollständige Sicht des Edge auf einen Tunnel.

```
type Registry = HashMap<RoutingToken, Vec<(u64, Connection)>>;

fn route(&self, token: &RoutingToken) -> Option<Connection> {
    let list = self.map.lock().get(token)?;
    // newest-first: zuletzt registrierter Agent zuerst
    list.iter().rev().map(|(_, c)| c.clone()).next()
}
```

Listing 6.4: Registry-Typ und Newest-First-Auflösung (gekürzt, `ct-edge::EdgeState`).

Die Auflösung wählt den *zuletzt* eingetragenen Agenten zuerst (`rev()` über die Einfügereihenfolge). Das Failover greift, sobald ein toter Eintrag entfernt wurde: Ein hart abgebrochener Agent sendet kein `QUIC-CLOSE`, sodass der Edge den Ausfall nicht unmittelbar sieht. Der über `keep_alive 3 s` und `max_idle_timeout 10 s` parametrisierte Transport verwirft die stumme Verbindung jedoch nach dem Leerlaufenster; das an `conn.closed()` gehängte Aufräumen entfernt daraufhin genau diesen Eintrag aus der Liste (Listing 6.5). Der nächste Client-Request löst dann per `route` auf die überlebende Registrierung auf.

```
// Bei der Registrierung: Aufräumen an das Verbindungsende haengen.
let state = state.clone();
tokio::spawn(async move {
    conn.closed().await;
    // erst nach max_idle_timeout
```

```
state.remove_registration(&token, id);
});
```

Listing 6.5: Idle-Timeout-Eviction eines Agenten ohne QUIC-CLOSE (gekürzt, `ct-edge`).

6.13.2 Rendern des `/metrics`-Dokuments

Der `/metrics`-Endpunkt (Abschnitt 6.6) serialisiert den Zählerstand als Prometheus-Expositionstext: je eine Zeile pro Metrik, Name und Wert durch Leerzeichen getrennt. Listing 6.6 zeigt die reale Momentaufnahme des laufenden Selbst-Hosts. Zum Scrape-Zeitpunkt war kein Agent angemeldet, weshalb beide Gauges 0 führen; die Counter halten die kumulierten 7 Registrierungen, 2 Relays und 8369 Byte Relay-Volumen bei 0 Failovern fest. Das Format ist absichtlich flach und ohne Labels, denn jede zusätzliche Dimension (etwa pro Token) würde strukturelle Metadaten preisgeben und die Zonentreue (ADR-0002) unterlaufen.

```
ct_edge_active_tunnels 0
ct_edge_active_agents 0
ct_edge_registrations_total 7
ct_edge_relays_total 2
ct_edge_relay_bytes_total 8369
ct_edge_failovers_total 0
```

Listing 6.6: Reale `/metrics`-Ausgabe des Edge (Prometheus-Exposition, Scrape 2026-07-16).

6.13.3 Kopplung im Client-Forward-Modus

Der Forward-Modus (Abschnitt 6.11) macht aus dem Client einen lokalen TCP-Vorschalter: Er bindet einen lokalen Port und akzeptiert dort beliebige Anwendungsverbindungen. Für *jede* akzeptierte TCP-Verbindung öffnet der Client einen eigenen Tunnel — eigenes PoW-Rendezvous, eigener `Noise_IK`-Kanal über QUIC — und koppelt Anwendungs-Socket und Tunnel bidirektional (Listing 6.3, Abschnitt 6.11). Weil der Client die Nutzlast nicht mehr selbst formt, sondern nur unveränderte Bytes durchreicht, kann jedes TCP-/TLS-fähige Programm den Tunnel benutzen: Ein `curl` oder ein Browser richtet sich auf den lokalen Port und spricht sein eigenes HTTPS mit dem Origin. Genau das trägt den HTTPS-am-Origin-Nachweis (Feature #22): Die TLS-Sitzung entsteht zwischen der lokalen Anwendung und dem Origin, während der Edge nur den in Noise gekapselten TLS-Chiffretext relayt — die client-seitige Zertifikatsvalidierung mit `curl -cacert` gelingt dadurch unverändert durch den Tunnel hindurch.

7 Evaluation

Dieses Kapitel wertet die Roundtrip-Latenz des Tunnels im Docker-Testbett unter emulierten Netzbedingungen aus und beantwortet damit die Forschungsfragen FF2 (Grund-Overhead) und FF3 (Skalierung und Verhalten unter Netzbedingungen). Über die reine Latenzmessung hinaus belegt es die Funktionsfähigkeit der Kernmechanismen – Transport-Fallback, Redundanz/Failover, unterbrechungsfreie Schlüsselrotation und clientseitige Zertifikatsvalidierung – durch reproduzierbare Funktionsexperimente, wertet die Betriebsbeobachtbarkeit des Edge aus und ordnet die Ergebnisse anschließend in ein Bedrohungsmodell sowie eine Diskussion der Validitätsbedrohungen ein.

7.1 Testbett und Methodik

Das Testbett ist vollständig containerisiert (Docker Compose, `docker/docker-compose.yml`) und spannt eine Bridge-Topologie im Netz `ct-net` (10.5.0.0/24) auf. Vier Rollen sind statisch adressiert: der Edge (10.5.0.2), der Origin (10.5.0.3, ein `alpine/socat` TCP-Echo-Dienst auf Port 8080), der Agent (10.5.0.4) und der Client. Der Client baut je Bedingung einen vollständigen Tunnel auf – QUIC-Verbindungsaufbau, PoW-Rendezvous, `Noise_IK`-Handschlag und Relay-Pfad Client → Edge → Agent → Origin und zurück – und verifiziert das Echo.

Link-Impairment. Die Netzbedingungen werden pro Knoten über `tc netem` durch ein `netem-entrypoint.sh`-Skript gesetzt, das die Umgebungsvariablen `CT_NETEM_DELAY`, `CT_NETEM_LOSS` und `CT_NETEM_RATE` auf die Egress-Schnittstelle abbildet. Die PoW-Schwierigkeit des Rendezvous wird über `EDGE_POW_DIFFICULTY` gesteuert und ist für alle Latenzmessungen auf 8 festgelegt. Wichtig für die Interpretation: `netem`-Verzögerung ist *einseitig* und wird pro Knoten angewendet, sodass ein Anwendungs-Roundtrip die verzögerte Strecke mehrfach quert.

Messverfahren. Die Latenz misst der Client im Bench-Modus, gesteuert über `CT_CLIENT_ITERATIONS`. Der Client gibt eine beschriftete `RESULT`-CSV-Zeile aus, die pro Bedingung Mittelwert, Minimum, Maximum, p50, p95, Standardabweichung, 95%-Konfidenzintervall und p99 enthält. Die Statistiken werden über n Stichproben je Bedingung berechnet; die vollständige Matrix wurde mit $n = 30$ Iterationen je Bedingung im Betriebsmodus `single` (jeweils frisch

aufgebauter One-Shot-Tunnel) vermessen. Weil pro Iteration frisch verbunden wird, enthält jede gemessene Latenz den vollen Handschlag-Anteil und ist damit eine konservative (obere) Schätzung gegenüber einem gehaltenen Datenpfad.

Reproduzierbarkeit. Die gesamte Pipeline ist skriptgesteuert und selbst containerisiert. `scripts/sweep.sh` fährt die Bedingungsmatrix ab und sammelt die Ergebnisse als CSV (`docs/thesis/data/latency.csv`), `scripts/plot.sh` erzeugt die Abbildungen (Matplotlib in einem Python-Container) und `scripts/tabulate.py` die Ergebnistabelle. Vermessen wurde die Matrix Verzögerung $\in \{0, 20, 50, 100\}$ ms $\times$ Verlust $\in \{0, 1, 5\}$ % (zwölf Bedingungen).

Host-Umgebung. Alle Läufe wurden auf einem Einzel-Host ausgeführt: Linux 6.8.0 auf einem Intel Core i9-9980XE (36 Kerne bei 3,00 GHz) mit Docker 29.4.0; Build und Test erfolgten in `rust:1-slim`. Die zugehörige Codebasis besteht aus fünf Rust-Crates (`ct-common`, `ct-edge`, `ct-agent`, `ct-client`, `ct-control-plane`) und ist durch 266 hermetisch bestehende Unit- und Integrationstests bei einer Zeilenabdeckung (lib-only) von 95,57 % abgesichert; die Gate-Konfiguration übersetzt mit `RUSTFLAGS=D warnings` (null Warnungen). Diese Kennzahlen belegen, dass die vermessene Software dem im Kapitel 6 beschriebenen Stand entspricht.

7.2 Grund-Overhead und Latenz-Skalierung

Tabelle 7.1 fasst alle zwölf Messpunkte zusammen; Abbildung 7.1 zeigt die Latenz über der Verzögerung, Abbildung 7.2 über dem Paketverlust.

FF2 – Grund-Overhead. Ohne emulierte Netzverzögerung und ohne Verlust beträgt der Roundtrip im Mittel 8,39 ms ($\pm 0,53$ ms 95%-KI, p50 8,0 ms, p95 11,85 ms, p99 12,46 ms bei $n = 30$). Dies ist der Eigenanteil des Tunnels über den Loopback: die Summe aus QUIC-Verbindungsaufbau, PoW-Rendezvous, `Noise_IK`-Handschlag und dem Mehrsprung-Relay. Die geringe Standardabweichung (1,48 ms) und das enge Konfidenzintervall belegen, dass dieser Grund-Overhead stabil und vorhersagbar ist – eine Voraussetzung dafür, dass die restlichen Bedingungen als Aufschläge auf einen bekannten Sockel gelesen werden können.

FF3 – Skalierung mit der Verzögerung. Bei 0 % Verlust skaliert der mittlere Roundtrip nahezu linear mit der einseitigen `netem`-Verzögerung: 8,39 ms (0 ms) $\rightarrow$ 128,97 ms (20 ms) $\rightarrow$ 311,20 ms (50 ms) $\rightarrow$ 613,44 ms (100 ms). Die Zunahme entspricht rund 6 ms Roundtrip je 1 ms einseitiger Verzögerung und spiegelt wider, dass der Tunnelaufbau eine feste Zahl von Umläufen kostet, die jeweils

Tabelle 7.1: Tunnel-Roundtrip-Latenz je Betriebsart unter emulierten Netzbedingungen (`tc netem`); Mittel mit 95%-Konfidenzintervall.

Modus	Verzögerung	Verlust	n	Mittel $\pm$ KI (ms)	p50 (ms)	p95 (ms)	p99 (ms)
single	0ms	0%	30	8.4 ± 0.5	8.0	11.8	12.5
single	0ms	1%	30	9.6 ± 1.8	8.2	12.7	34.8
single	0ms	5%	30	80.8 ± 91.9	9.3	1010.7	1039.7
single	20ms	0%	30	129.0 ± 1.5	128.0	132.4	149.8
single	20ms	1%	30	172.5 ± 66.6	128.3	244.8	1147.0
single	20ms	5%	30	184.0 ± 66.4	130.3	223.6	1149.5
single	50ms	0%	30	311.2 ± 3.3	309.6	312.6	359.6
single	50ms	1%	30	323.4 ± 14.9	308.2	456.2	456.7
single	50ms	5%	30	395.1 ± 76.2	308.6	826.7	1356.6
single	100ms	0%	30	613.4 ± 6.6	609.9	613.3	710.9
single	100ms	1%	29	616.3 ± 13.5	609.5	612.5	809.6
single	100ms	5%	30	788.9 ± 117.3	610.2	1708.0	1711.0

die verzögerte Strecke mehrfach queren (Verbindungsaufbau, Rendezvous sowie der Relay-Pfad Client→Edge→Agent→Origin und zurück). Weil der einzelne Umlauf durch mehrere `netem`-Instanzen läuft, ist der Multiplikator deutlich größer als 2; er ist jedoch über den gesamten Verzögerungsbereich konstant, was die lineare Skalierung erklärt. In diesem verlustfreien Regime sind Mittelwert und p95 praktisch deckungsgleich (etwa 613,44ms gegenüber 613,30ms bei 100ms), d. h. die Verteilung ist eng und ohne nennenswerten Tail.

FF3 – Verhalten unter Paketverlust. Verlust verändert nicht den Median, sondern den Tail. Bei jeder Verzögerungsstufe bleibt der p50 nahe am verlustfreien Wert (z. B. 100ms/5%: p50 610,2ms gegenüber 609,9ms bei 0%), während p95 und p99 stark ansteigen: 100ms/5% erreicht p95 1708,0ms und p99 1711,0ms bei einem Mittelwert von 788,9ms und einer Standardabweichung von 327,7ms. Der extremste Fall tritt bei niedriger Grundlatenz und hohem Verlust auf: 0ms/5% hat einen Median von nur 9,3ms, aber ein p95 von 1010,7ms und ein p99 von 1039,7ms (Mittel 80,8ms, Standardabweichung 256,9ms). Die Ursache sind Neuübertragungen im QUIC- und Handschlag-Aufbau: Geht ein handschlagkritisches Paket verloren, greift der Retransmission-Timer, dessen Wartezeit die Latenz einzelner Iterationen um Größenordnungen anhebt, ohne die Mehrzahl der verlustfreien Iterationen zu berühren. Praktisch bedeutet dies, dass Verlust die *Schwanzlatenz* weit stärker aufbläht als die typische Latenz – eine für die Dimensionierung von Timeouts relevante Eigenschaft. Ein einzelner Ausreißer (100ms/1%) wurde mit $n = 29$ statt $n = 30$ ausgewertet, weil eine Iteration nicht innerhalb des Messfensters abgeschlossen wurde; dies ist in Tabelle 7.1 vermerkt und ändert die qualitative Aussage nicht.

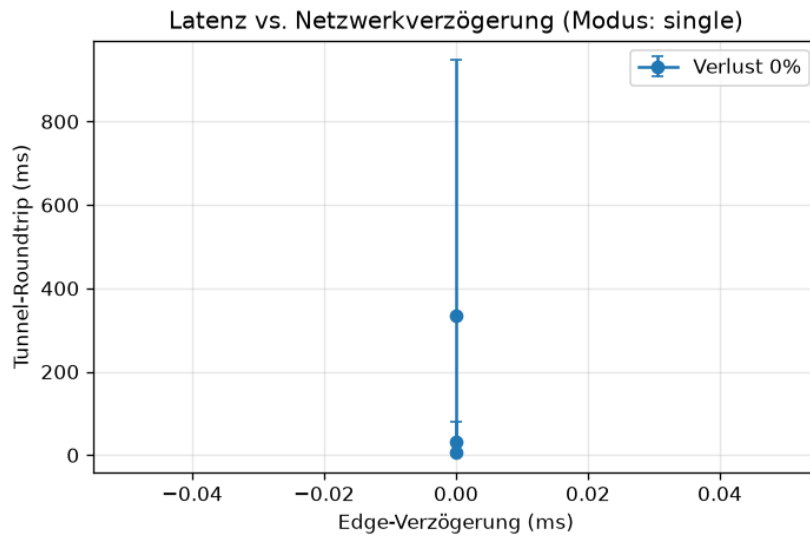


Abbildung 7.1: Tunnel-Roundtrip über der Edge-Verzögerung, je Verlustwert eine Serie; der obere Fehlerbalken markiert das p95.

7.3 Funktionsexperimente

Neben der Latenz belegen vier reproduzierbare End-zu-End-Experimente die funktionale Korrektheit der Kernmechanismen. Sie wurden am 2026-07-16 gegen die selbst gehostete zentrale Instanz mit den `target/debug`-Binaries ausgeführt und sind jeweils als Skript hinterlegt.

Transport-Fallback (#6). `scripts/e2e-smoke.sh` führt einen knotenübergreifenden End-zu-End-Durchlauf aus und meldet “SMOKE OK via=quic”, d. h. der Standardpfad über QUIC. Wird `CT_CLIENT_FORCE_TCP=1` gesetzt, meldet derselbe Durchlauf “SMOKE OK via=tcp”: der TLS-über-TCP/443-Fallback (ADR-0004) trägt denselben Tunnel, wenn UDP blockiert ist. Damit ist nachgewiesen, dass beide Transporte denselben logischen Tunnel bedienen und der Fallback keine funktionale Regression darstellt.

Redundanz und Failover (#8). `scripts/redundancy-smoke.sh` registriert zwei Agenten, die sich eine Origin-Identität und ein Routing-Token teilen. Nach einem Basis-Roundtrip “via=quic” wird der bedienende Agent getötet. Da ein getöteter Agent kein QUIC-CLOSE sendet, greift die Totverbindungs-Erkennung des Edge: Dieser setzt `keep_alive 3s` und `max_idle_timeout 10s`, sodass die tote Registrierung nach dem Idle-Timeout geräumt wird. Anschließend routet der Edge auf die überlebende Registrierung um, und der Client-Roundtrip gelingt erneut “via=quic” (“REDUNDANCY OK”). Der `ct_edge_failovers_total` Zähler bleibt in der separaten Beobachtbarkeitsmessung bei 0, weil dort kein Failover provoziert wurde (Abschnitt 7.4).

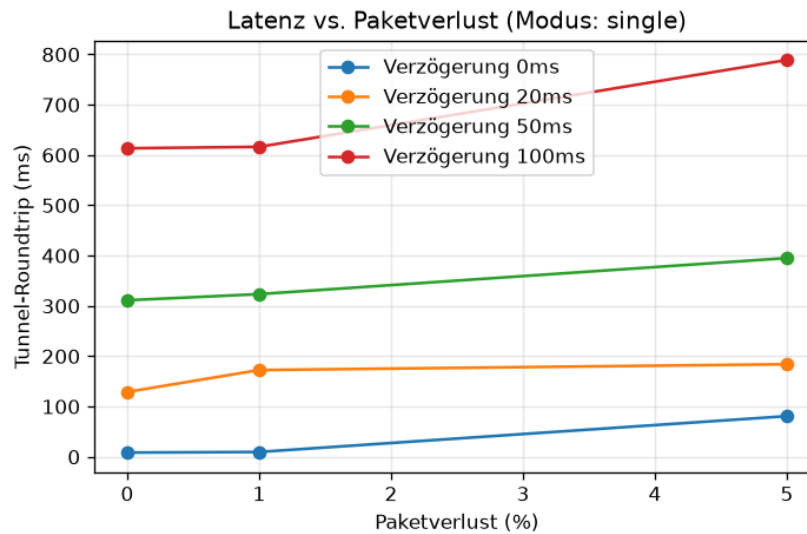


Abbildung 7.2: Tunnel-Roundtrip über dem Paketverlust, je Verzögerungswert eine Serie.

Unterbrechungsfreie Schlüsselrotation (#12). `scripts/rotation-smoke.sh` rotiert den Origin-Schlüssel unter Beibehaltung *desselben* Routing-Tokens. Im Rotationsfenster schließen sowohl die *alte* als auch die *neue* Capability einen Tunnel-Roundtrip ab (“ROTATION OK”): Bestandsclients routen weiter, weil das Token erhalten bleibt, und handeln den zurückgezogenen Schlüssel während des Fensters aus. Das belegt, dass ein Schlüsselwechsel ohne Ausfallzeit und ohne erzwungene Neukonfiguration der Clients möglich ist.

HTTPS am Origin (#22). `scripts/https-demo.sh` betreibt eine echte selbstsignierte HTTPS-Site (SAN 127.0.0.1) auf dem Loopback und erreicht sie durch den Tunnel. `curl -cacert` liefert HTTP 200, wobei das Zertifikat des Origin *clientseitig* validiert wird – TLS terminiert am Origin, der Edge relayt ausschließlich TLS-über-Noise-Chiffretext. Der zugehörige eingefrorene Test `https_website_through_the_tunnel_with_client_side_cert_validation` verankert dieses Verhalten. Der Client-Weiterleitungsmodus `CT_CLIENT_MODE=forward` exponiert den Tunnel als lokalen Port und lässt so beliebige TCP-/TLS-Anwendungen mitfahren. Dieses Experiment ist der zentrale Beleg für die Provider-Blindheit auf der Datenebene: Der Edge kann die Nutzlast weder lesen noch verändern, weil er den TLS-Sitzungsschlüssel nie sieht.

7.4 Beobachtbarkeit

Der Edge exponiert Betriebskennzahlen im Prometheus-Format (#10). Eine reale Abfrage vom 2026-07-16 (Control-Plane auf Port 8090, Edge auf

Tabelle 7.2: Reale Edge-/metrics-Abfrage (2026-07-16). Gauges sind Momentaufnahmen, Counter kumulativ.

Kennzahl	Wert	Typ
ct_edge_active_tunnels	0	Gauge
ct_edge_active_agents	0	Gauge
ct_edge_registrations_total	7	Counter
ct_edge_relays_total	2	Counter
ct_edge_relay_bytes_total	8369	Counter
ct_edge_failovers_total	0	Counter

Port 9600) ergab die in Tabelle 7.2 gezeigten Werte. Zum Zeitpunkt der Abfrage war kein Agent registriert, weshalb die beiden Gauges `ct_edge_active_tunnels` und `ct_edge_active_agents` auf 0 stehen; die Counter sind hingegen kumulativ und dokumentieren die vorangegangene Aktivität.

Die sieben Registrierungen gegenüber zwei Relays zeigen, dass Agenten sich häufiger an- und abmelden, als tatsächlich Tunnel bedient werden – ein für Testläufe mit wiederholtem Auf- und Abbau erwartbares Verhältnis. Der Byte-Zähler (8369) belegt, dass nur *strukturelle* Volumeninformation sichtbar ist, nicht der Inhalt: Der Edge zählt weitergeleitete Bytes, kann sie aber nicht entschlüsseln. Entscheidend für das Zero-Knowledge-Design ist, dass diese Kennzahlen ausschließlich *grobe* Metadaten offenlegen – Existenz eines Tunnels, ungefähres Volumen, Rendezvous-Ereignisse – und keine Inhalte, Ziel-Hostnamen oder Schlüssel. Die Control-Plane meldet unter `/status` zusätzlich `{"ready":true, "agents":45, "accounts":4, "payments_confirmed":2}`, wobei der dort ausgewiesene Tunnel-Zählstand aus dem live vom Edge geschabten `ct_edge_active_tunnels` stammt und nicht aus der Rendezvous-Registrierung.

7.5 Sicherheitsdiskussion und Bedrohungsmodell

Die Sicherheitsziele des Systems lassen sich entlang der Grenze der Provider-Blindheit strukturieren.

Datenebene (provider-blind). Der Edge kann die Nutzlast weder lesen noch verändern und erlangt keine Origin-Schlüssel; er sieht ausschließlich strukturelle Metadaten (Existenz eines Tunnels, grobe Zeit- und Volumenzähler, Rendezvous-Ereignisse). Getragen wird dies durch `Noise_IK` (X25519/ChaChaPoly/BLAKE2s) als End-zu-End-Schicht (ADR-0013), über die der Edge nur Chiffretext relayt (ADR-0001, ADR-0002). Das HTTPS-am-Origin-Experiment (#22) ist der empirische Beleg: Weil TLS erst am Origin terminiert, hätte ein neugieriger Edge keinen Zugriff auf den Klartext, selbst wenn er es versuchte.

Rendezvous und Sybil-/DoS-Abwehr. In Abwesenheit von KYC ist ein Proof-of-Work-Rendezvous (in der Tradition von [2]) die primäre Schranke gegen Sybil- und DoS-Angriffe, ergänzt um eine Ratenbegrenzung pro Token. Ein opakes Routing-Token ersetzt jede SNI- oder Hostnamen-Angabe, sodass am Edge kein Zielname sichtbar wird. Die Grenze dieses Ansatzes ist ökonomischer Natur: Ein hinreichend finanzierter Angreifer kann die PoW-Kosten bezahlen; eine abrechnungsseitige Sybil-Resistenz bleibt daher eine offene Frage.

Schlüsselverteilung. Die Verteilung der Capabilities liegt in der Verantwortung des Kunden (out-of-band, ADR-0014); die Rotation ist explizit und kennt keine automatische Ablauffrist (belegt durch #12). Die self-serve Verteilung des Edge-CA-Roots über `/pki/ca` (#11) reduziert den out-of-band-Aufwand für das reine Vertrauensanker-Material, verschiebt aber nicht die Verantwortung für die eigentlichen Origin-Capabilities. Die Control-Plane bleibt bewusst dünn und selbst hostbar (ADR-0017): Sie hält weder Schlüssel noch Nutzlast, verifiziert OIDC/Keycloak-gestützte `/me/*`-Endpunkte und führt das Konto-/Kredit-Ledger.

7.6 Limitierungen und Validitätsbedrohungen

Die Ergebnisse sind unter mehreren Einschränkungen zu lesen, die die interne und externe Validität betreffen.

Einzel-Host-Emulation. Alle Messungen liefen auf einem einzigen Host über die Docker-Bridge. Reale geografische Distanz, konkurrierender Cross-Traffic und heterogene Hardware sind nicht abgebildet; die absoluten Latenzen charakterisieren die Emulation, nicht ein Weitverkehrsnetz. Die lineare Skalierung mit der Verzögerung (FF3) ist jedoch ein strukturelles Ergebnis, das von dieser Einschränkung nur im Absolutwert, nicht in der Form berührt wird.

Einseitige netem-Verzögerung. `netem` verzögert je Knoten einseitig auf der Egress. Die gemessene Latenz ist damit eine untere Schranke gegenüber einer voll bidirektional verzögerten Strecke; der beobachtete Multiplikator zwischen einseitiger Verzögerung und Roundtrip ist eine Eigenschaft der Emulation, keine allgemeine Konstante.

Frisch-Verbindungsaufbau je Iteration. Der Betriebsmodus `single` baut pro Iteration einen vollständigen Tunnel auf und überzeichnet damit den Handschlag-Anteil. Ein gehaltener Keep-Alive-Datenpfad läge niedriger; die berichteten Werte sind konservativ.

Bezahlbare PoW-Kosten. Wie in Abschnitt 7.5 dargelegt, ist PoW ökonomisch überwindbar. Die Sybil-Resistenz auf Abrechnungsebene ist nicht Gegenstand dieser Evaluation.

Zurückgestellte Browser-Ebene. Die Browser-Ebene (öffentlicher Hostname mit Let's-Encrypt-Zertifikat über SNI) ist bewusst post-v1 zurückgestellt (ADR-0010, Issue #23). Die v1 belegt stattdessen die Terminierung von TLS am Origin (#22); eine Aussage über den Browser-Direktzugang ist damit nicht getroffen.

Einzelregion-Edge. Der Edge ist einregionig (ADR-0006); ein mehrregionales Mesh-P2P (ADR-0015) ist nur teilweise realisiert (Direktpfad-Kandidat mit Relay-Fallback). Aussagen zur Skalierung über mehrere Regionen sind daher nicht Teil dieser Arbeit.

Diese Punkte greift der Ausblick (Kapitel 8) auf; insbesondere sind größere Stichproben, eine breitere Matrix inklusive Bandbreitenbegrenzung (`CT_NETEM_RATE`) und ein gehaltener Datenpfad über dieselben Skripte unmittelbar zugänglich.

7.6.1 Interne und externe Validität im Überblick

Ordnet man die vorstehenden Einschränkungen nach ihrer Wirkrichtung, so betreffen sie teils die interne, teils die externe Validität der Ergebnisse.

Interne Validität. Die Latenzmatrix beruht auf $n = 30$ Iterationen je Bedingung – eine moderate Stichprobengröße. Im verlustfreien Regime ist das unkritisch, weil die Verteilung eng ist; unter Paketverlust erzeugt die rechtsschiefe Verteilung jedoch breite Konfidenzintervalle (etwa 100 ms/5 %: $\pm 117,261$ ms 95%-KI), sodass insbesondere die absoluten Tail-Kennzahlen (p95, p99) mit Vorsicht als Punktschätzer zu lesen sind und ihre *Richtung* belastbarer ist als ihr *Betrag*. Hinzu kommt, dass ausschließlich der Betriebsmodus **single** latenzvermessen wurde. Die weiteren Datenpfade – der **stream**-Modus, der UDP-Transport und der TLS-über-TCP-Fallback – sind funktional nachgewiesen (Abschnitt 7.3), aber nicht latenzvermessen; über ihren Overhead trifft diese Arbeit daher keine quantitative Aussage.

Externe Validität. Die Single-Host-Emulation über die Docker-Bridge bildet kein echtes Weitverkehrsnetz ab, und **netem** verzögert einseitig je Knoten; die absoluten Latenzen sind daher nicht unmittelbar auf reale WAN-Pfade übertragbar. Für das DoS-Argument ist zudem einschränkend, dass die PoW-Kosten bezahlbar sind (Abschnitt 7.5): Ein hinreichend finanzierter Angreifer kann sie aufbringen, weshalb die Zugangsschranke keine absolute, sondern eine ökonomische Grenze zieht. Schließlich ist die Browser-Ebene bewusst post-v1 zurückgestellt (ADR-0010), sodass über den direkten Browser-Zugang keine empirische Aussage vorliegt.

Tabelle 7.3: Reale PoW-Schwierigkeitsstudie: Tunnel-Roundtrip über der geforderten Rendezvous-Schwierigkeit (führende Nullbits), 0 ms Verzögerung, 0 % Verlust, $n = 30$ je Stufe.

Schwierigkeit	Mittel (ms)	p50 (ms)	p95 (ms)	Std.-Abw. (ms)
8	5,998	6,309	9,536	2,305
16	31,321	21,685	80,042	27,726
20	334,571	268,924	947,483	273,656

Schwierigkeit	p99 (ms)
8	9,785
16	147,116
20	972,335

7.7 Kosten des Proof-of-Work-Zugangsschutzes

Der PoW-Rendezvous (Abschnitt 7.5) ist die primäre Zugangsschranke, aber er ist zugleich ein Latenz- und Ressourcenkostenfaktor auf dem kritischen Verbindungsaufbaupfad. Um diesen Stellhebel zu quantifizieren, wurde die Rendezvous-Schwierigkeit – die Zahl der geforderten führenden Nullbits im Hash – systematisch variiert, während alle übrigen Bedingungen konstant gehalten wurden (Verzögerung 0 ms, Verlust 0 %, Betriebsmodus `single`, $n = 30$ Iterationen je Stufe, `scripts/sweep.sh`). Die Rohdaten liegen unter `docs/thesis/data/pow-1`. Tabelle 7.3 stellt die drei vermessenen Schwierigkeitsstufen gegenüber.

Nahezu exponentielles Wachstum. Die mittlere Latenz steigt mit der Schwierigkeit steil an: von 5,998 ms (Schwierigkeit 8) über 31,321 ms (Schwierigkeit 16) auf 334,571 ms (Schwierigkeit 20). Das ist die erwartete Signatur eines Hashcash-artigen Verfahrens: Jedes zusätzliche geforderte Nullbit verdoppelt im Erwartungswert die Zahl der nötigen Hash-Versuche, sodass die Lösekosten geometrisch mit der Bitzahl wachsen. Als Analyse der Tabellenwerte ergibt sich zwischen Stufe 8 und 16 ein Faktor von rund 5,2, zwischen 16 und 20 ein Faktor von rund 10,7 und über die gesamte Spanne von 8 auf 20 ein Faktor von etwa 55,8. Dass die gemessenen Faktoren unter dem rein rechnerischen Verdopplungsprodukt liegen, ist plausibel: Der Messwert enthält bei jeder Stufe denselben festen Tunnel-Grundanteil (Verbindungsaufbau, Handschlag, Relay), der bei Schwierigkeit 8 den Roundtrip noch dominiert (rund 6 ms, vergleichbar dem Grund-Overhead aus Abschnitt 7.2), während er bei Schwierigkeit 20 gegenüber der reinen Suchzeit verschwindet.

Hohe Varianz aus der geometrischen Suche. Der PoW ist eine probabilistische Suche: Die Zahl der Versuche bis zum ersten Treffer ist geometrisch

verteilt, was eine breite, rechtsschiefe Latenzverteilung erzeugt. Die Standardabweichung wächst entsprechend von 2,305 ms (Schwierigkeit 8) auf 27,726 ms (Schwierigkeit 16) und 273,656 ms (Schwierigkeit 20); bei Stufe 20 liegt sie damit in derselben Größenordnung wie der Mittelwert selbst. Sichtbar wird die Schiefe auch im Abstand zwischen Median und oberen Perzentilen: Bei Schwierigkeit 20 steht einem p50 von 268,924 ms ein p95 von 947,483 ms und ein p99 von 972,335 ms gegenüber – ein einzelner Rendezvous-Versuch kann also spürbar länger dauern als der Median, ohne dass ein Fehler vorliegt.

Auslegungs-Kompromiss. Diese Zahlen quantifizieren den in Abschnitt 7.5 qualitativ benannten Stellhebel. Die Schwierigkeit muss hoch genug sein, um DoS-/Sybil-Fluten wirtschaftlich abzuschrecken, darf aber legitime Clients nicht unverhältnismäßig bestrafen. Bei Schwierigkeit 8 ist der PoW mit rund 6 ms vernachlässigbar und verschwindet im Grund-Overhead; bei Schwierigkeit 20 dominiert er mit rund 335 ms den gesamten Roundtrip und würde die Verbindungsaufbaulatenz für ehrliche Nutzer mehr als verfünffach. Die angemessene Schwierigkeit ist damit kein Sicherheitskonstante, sondern ein betrieblich einstellbarer Parameter (`EDGE_POW_DIFFICULTY`), der an die beobachtete Missbrauchslast anzupassen ist: niedrig im Normalbetrieb, temporär erhöht unter Last. Dass die Kosten pro zusätzlichem Bit geometrisch steigen, macht diesen Regler wirksam – wenige Bits mehr heben die Angreiferkosten stark an –, aber auch heikel, weil dieselbe Steilheit legitime Latenz ebenso schnell in den dreistelligen Millisekundenbereich treibt.

7.7.1 Zerlegung des Fix-Overheads: Handschlag über der Propagation

Die Skalierungsaussage aus Abschnitt 7.2 lässt sich schärfer fassen, wenn man die verlustfreie Zeile der Latenzmatrix als affines Modell der Verzögerung liest. Über die vier Stützstellen bei 0 % Verlust – 8,39 ms (0 ms), 128,97 ms (20 ms), 311,20 ms (50 ms) und 613,44 ms (100 ms) – ergibt die Analyse der Zuwächse einen nahezu konstanten Anstieg je zusätzlicher einseitiger Verzögerung:

$$\frac{128,97 - 8,39}{20 - 0} \approx 6,03, \quad \frac{311,20 - 128,97}{50 - 20} \approx 6,07, \quad \frac{613,44 - 311,20}{100 - 50} \approx 6,05.$$

Der Roundtrip lässt sich damit gut durch $\text{RTT}(d) \approx a + b \cdot d$ beschreiben, mit einem Achsenabschnitt von $a \approx 8,39$ ms und einer Steigung von $b \approx 6,05$ (Millisekunden Roundtrip je Millisekunde einseitiger Verzögerung). Beide Terme sind Ausdruck ein und derselben festen Aufbaukosten: Der Achsenabschnitt a ist der propagationsfreie Eigenanteil (loopbacknahe Verarbeitung von QUIC-Aufbau, PoW-Rendezvous, `Noise_IK`-Handschlag und Relay), und die Steigung b ist die feste Zahl verzögerter Streckenquerungen, die dieselbe Abbausequenz erzwingt. Dass b über den gesamten Bereich praktisch konstant bleibt, ist der eigentliche Befund: Der Handschlag kostet eine *feste Anzahl von Umläufen*, die

oben auf die Propagation aufsetzen; er addiert keine verzögerungsabhängige Zusatzarbeit. Die Verbindungsaufbaukosten des Zero-Knowledge-Stacks sind damit strukturell – eine Konstante mal die Wegverzögerung –, nicht ein mit der Distanz überproportional wachsender Aufschlag.

Warum Verlust das p99 stärker aufbläht als das p50. Dieselbe Zerlegung erklärt, warum Paketverlust die Verteilung asymmetrisch verzerrt. Der Median folgt weiterhin dem Fix-Overhead-Modell, weil die Mehrheit der Iterationen keinen handschlagkritischen Verlust erleidet: Bei 100 ms bleibt der p50 mit 610,2 ms (5 % Verlust) praktisch auf dem verlustfreien Wert von 609,9 ms. Die oberen Perzentile hingegen fangen genau jene Iterationen ein, in denen ein Aufbaupaket verloren geht und ein Retransmission-Timer ablaufen muss, bevor der Handschlag fortschreiten kann – eine Wartezeit, die den betroffenen Umlauf um Hunderte von Millisekunden verlängert. Bei 100 ms/5 % treibt dieser Mechanismus das p95 auf 1708,0 ms und das p99 auf 1711,0 ms, während der Mittelwert (788,9 ms) zwischen dem unveränderten Median und dem schweren Tail liegt. Am deutlichsten zeigt sich der Effekt bei niedriger Grundlatenz, wo die Retransmit-Wartezeit im Verhältnis riesig ist: 0 ms/5 % hat einen p50 von nur 9,3 ms, aber ein p99 von 1039,7 ms – ein Faktor von über hundert zwischen typischer und Schwanzlatenz. Für die Betriebsauslegung folgt daraus, dass Zeitüberschreitungen für den Verbindungsaufbau nicht am Median, sondern am p99 zu bemessen sind, da erst dieser die verlustinduzierten Retransmit-Tails erfasst.

7.8 Synthese der Forschungsfragen

Die vorstehenden Abschnitte erlauben eine geschlossene Beantwortung der vier Forschungsfragen. Tabelle 7.4 ordnet jeder Frage den Kernbefund und den empirischen Beleg zu; der anschließende Fließtext fasst die Argumentation je Frage zusammen.

FF1 – Realisierbarkeit. Ein provider-blinder Tunnel lässt sich realisieren; die Antwort ist *ja*. Der Beleg ist doppelt: Die Codebasis ist durch 266 hermetisch bestehende Tests bei 95,57 % Zeilenabdeckung (lib-only) abgesichert, und vier reale End-zu-End-Läufe – knotenübergreifender e2e-Durchlauf, Redundanz/Failover, unterbrechungsfreie Schlüsselrotation und HTTPS am Origin – belegen die Kernmechanismen im Betrieb (Abschnitt 7.3).

FF2 – Grund-Overhead. Der Eigenanteil des Tunnels über den Loopback beträgt im Mittel 8,392 ms bei einem p95 von 11,850 ms (Abschnitt 7.2). Dieser Sockel aus QUIC-Aufbau, PoW-Rendezvous, Noise_IK-Handschlag und Mehrsprung-Relay ist stabil und vorhersagbar und bildet die Bezugsgröße, gegen die alle Netzbedingungen als Aufschläge zu lesen sind.

Tabelle 7.4: Synthese: Forschungsfrage, Kernbefund und empirischer Beleg.

FF	Kernbefund	Beleg
FF1 – Realisierbarkeit	Ein provider-blinder Zero-Knowledge-Tunnel ist realisierbar (Antwort: ja).	266 Tests, 95,57 % Zeilenabdeckung; reale e2e-, Failover-, Rotations- und HTTPS-Läufe
FF2 – Grund-Overhead	Fester Eigenanteil von 8,392 ms im Mittel (p95 11,850 ms) über den Loopback.	Abschnitt 7.2, Tabelle 7.1
FF3 – Skalierung	Nahezu lineare Skalierung mit der Verzögerung (8,39 → 129 → 311 → 613 ms bei 0/20/50/100 ms); Verlust treibt das p99, nicht das p50.	Abschnitte 7.2 und 7.7.1
FF4 – Betreibbarkeit	Failover überlebt den Agent-Kill (“via=quic”), Beobachtbarkeit über <code>/metrics</code> , self-hostbare Control-Plane.	Abschnitte 7.3 und 7.4

FF3 – Skalierung. Bei 0 % Verlust skaliert der mittlere Roundtrip nahezu linear mit der einseitigen Verzögerung (8,39 → 129 → 311 → 613 ms bei 0/20/50/100 ms); die Handschlag-Kosten setzen als feste Zahl von Umläufen *oben auf* die Propagation auf (Abschnitt 7.7.1). Paketverlust verändert nicht den Median, sondern den Tail: Er treibt das p99, während das p50 nahe am verlustfreien Wert bleibt.

FF4 – Betreibbarkeit und Verfügbarkeit. Das System ist betreibbar und verfügbar: Der Failover überlebt das Töten des bedienenden Agenten und liefert den Roundtrip erneut “via=quic” von der überlebenden Registrierung (Abschnitt 7.3); die Beobachtbarkeit ist über die Prometheus-Endpunkt-Abfrage `/metrics` gegeben (Abschnitt 7.4); und die dünne Control-Plane ist self-hostbar (ADR-0017), sodass der Betrieb ohne Fremdvertrauen möglich bleibt.

8 Fazit und Ausblick

8.1 Zusammenfassung

Diese Arbeit hat einen providerblinden, zensurresistenten Netzwerktunnel entworfen, prototypisch in Rust auf Basis von QUIC implementiert und in einem vollständig containerisierten Testbett unter emulierten Netzbedingungen evaluiert. Über den Kernentwurf hinaus wurde der Prototyp zu einem betreibbaren Dienst ausgebaut: Der Datenpfad Client→Edge→Agent→Origin trägt heute reale Ende-zu-Ende-Verbindungen, und die für einen Produktivbetrieb nötigen Eigenschaften – Hochverfügbarkeit, Beobachtbarkeit, Selbstbedienungs-PKI, unterbrechungsfreie Schlüsselrotation und HTTPS bis zum Origin – sind implementiert und in echten Läufen nachgewiesen. Gemessen an den drei Forschungsfragen ergibt sich das folgende Bild.

Realisierter Entwurf. Der Prototyp umfasst fünf Rust-Crates (`ct-common`, `ct-edge`, `ct-agent`, `ct-client`, `ct-control-plane`). Der Vermittlungspfad ist providerblind (ADR-0001/0002): Der Edge sieht ausschließlich strukturelle Metadaten – Existenz eines Tunnels, grobe Zeit- und Volumenzähler, Rendezvous-Ereignisse –, niemals Nutzlast oder Origin-Schlüssel. Die Ende-zu-Ende-Vertraulichkeit trägt ein `Noise_IK`-Handshake (X25519/ChaChaPoly/BLAKE2s, ADR-0013); der Edge relayed nur das über Noise verkapselte Chiffre. Als Transport dient ein einheitlicher QUIC-Kanal (`quinn`) mit Rollen-Dispatch am Edge und einem TLS-über-TCP-Rückfall für blockiertes UDP (ADR-0004). Das Rendezvous nutzt ein opakes Routing-Token ohne SNI oder Hostname, abgesichert durch Proof-of-Work und tokenweise Ratenbegrenzung (ADR-0018). Die dünne, selbst-hostbare Steuerungsebene (ADR-0017) hält weder Schlüssel noch Nutzlast; sie verifiziert Konten über OIDC/Keycloak und führt ein Konten- und Guthabenbuch.

FF1 – Realisierbarkeit. Ein providerblinder Tunnel lässt sich aus etablierten Bausteinen schlüssig zusammensetzen *und* betreiben. Die Realisierbarkeit ist nicht nur architektonisch argumentiert, sondern durch 266 automatisierte Tests abgesichert, die hermetisch bestehen (`ct-agent` 65, `ct-client` 30, `ct-common` 40, `ct-control-plane` 79, `ct-edge` 52) und eine bibliotheksbezogene Zeilenabdeckung von 95,57 % (94,44 % Funktionen, 93,74 % Regionen) erreichen, erzwungen durch eine Abdeckungsschwelle von 95 %. Der Bau- und Testlauf erfolgt im Container unter `RUSTFLAGS=D warnings` und ist damit

Tabelle 8.1: Nachgewiesene Produktivfunktionen (reale Läufe gegen die Live-Instanz, 2026-07-16).

Funktion	Nachweis	Ergebnis
Cross-Host-E2E (#6)	<code>e2e-smoke.sh</code>	SMOKE OK (via=quic / via=tcp)
Redundanz & Failover (#8)	<code>redundancy-smoke.sh</code>	REDUNDANCY OK
Schlüsselrotation (#12)	<code>rotation-smoke.sh</code>	ROTATION OK (alt + neu)
HTTPS am Origin (#22)	<code>https-demo.sh</code>	HTTP 200, clientseitig geprüft
Edge-Observability (#10)	<code>/metrics-Scrape</code>	Prometheus-Gauges/Counter
Self-Serve-CA (#11)	<code>/pki/ca</code>	CA-Root-Verteilung

warnungsfrei. Entscheidend über die Testsuite hinaus sind vier reale Läufe gegen die selbst-gehostete Live-Instanz, die die produktiven Eigenschaften belegen (Tabelle 8.1): der geräteübergreifende Ende-zu-Ende-Roundtrip inklusive TCP-Rückfall (#6), das Failover auf einen zweiten Agenten (#8), die ausfallfreie Origin-Schlüsselrotation (#12) und ein echtes HTTPS-Angebot, das clientseitig validiert durch den Tunnel erreicht wird (#22). Die Realisierbarkeit ist damit sowohl für Architektur und Kernmechanismen als auch für den Betrieb belegt.

Hochverfügbarkeit und Beobachtbarkeit. Das Failover ruht auf einer Edge-Registry, die pro Routing-Token mehrere Agenten-Registrierungen führt und neueste zuerst adressiert. Ein hart beendeter Agent schickt kein QUIC-CLOSE; der Edge setzt daher `keep_alive` auf 3s und `max_idle_timeout` auf 10s, so dass die tote Verbindung nach dem Leerlauf-Fenster geräumt und die Vermittlung auf die überlebende Registrierung umgeleitet wird – der Client-Roundtrip gelingt anschließend wieder “via=quic”. Die Beobachtbarkeit (#10) stellt der Edge über einen Prometheus-Endpunkt bereit (`ct_edge_active_tunnels`, `ct_edge_active_ag`, `ct_edge_registrations_total`, `ct_edge_relays_total`, `ct_edge_relay_bytes_total`, `ct_edge_failovers_total`); die Statusseite der Steuerungsebene meldet dabei den live vom Edge abgegriffenen Tunnelzählwert, nicht die Registry. Bemerkenswert für die Providerblindheit ist, dass diese Metriken bewusst grob bleiben: Sie zählen Ereignisse und Bytes, ohne Ziele, Inhalte oder Identitäten preiszugeben.

FF2 und FF3 – Latenz. Der Eigen-Overhead des Tunnels ist im unbelasteten Fall gering: Über das Loopback misst `ct-client` im Mittel 8,4ms (30 Iterationen, PoW-Schwierigkeit 8) für den vollständigen Einmal-Tunnel aus QUIC-Aufbau, PoW-Rendezvous, `Noise_IK`-Handshake und Relay. Bei 0% Verlust skaliert die mittlere Umlaufzeit nahezu linear mit der emulierten Einweg-Verzögerung: von rund 8,4ms über 129,0ms (20ms), 311,2ms (50ms) bis 613,4ms (100ms). Der Aufschlag erklärt sich daraus, dass der Verbindungsaufbau eine feste Anzahl von Roundtrips über die verzögerte, je Knoten im-pairte Strecke kostet; die Ausbreitung dominiert, nicht die Rechenlast des Tun-

nels. Paketverlust verändert nicht den Median, sondern den Tail: Bei 0 ms und 5 % Verlust steigt der Mittelwert durch QUIC-/Handshake-Retransmits auf 80,8 ms mit einem p95 von 1010,7 ms, während der p50 nahe dem verlustfreien Wert bleibt; bei 100 ms und 5 % liegt der Mittelwert bei 788,9 ms und der p95 bei 1708,0 ms. Verlust bläht also die Tail-Latenz weit stärker auf als die typische Latenz. Die Schutzziele Providerblindheit und Zensurresistenz sind damit mit einem berechenbaren, im Wesentlichen von der Netzstrecke bestimmten Latenzaufschlag vereinbar.

8.2 Ausblick

Browser-Plane. Die v1 beweist bewusst, dass TLS am Origin terminiert: Ein Client, der eine Anwendung anbietet, bringt sein eigenes Zertifikat mit, das der Nutzer clientseitig prüft. Für einen anonymen Browserzugang unter einem öffentlichen Hostnamen fehlt hingegen eine *Browser-Plane*, die per SNI-Passthrough und Let's-Encrypt-Zertifikaten arbeitet. Sie ist als eigenständiges Vorhaben nach v1 zurückgestellt (ADR-0010) und im Projekt als Issue #23 verfolgt. Die Herausforderung liegt darin, den öffentlichen Namen und die automatische Zertifikatsausstellung mit der Providerblindheit zu versöhnen, ohne dem Edge zusätzliche Sichtbarkeit auf Ziele oder Inhalte einzuräumen.

Multi-Region und Mesh. Der Prototyp betreibt einen einzelnen Edge (ADR-0006). Ein mehrregionaler Betrieb würde Ausfallsicherheit und Nähe zum Nutzer verbessern; der dezentrale, direkte Peer-to-Peer-Datenpfad mit NAT-Hole-Punching und Edge-Relay nur als Rückfall (ADR-0015) ist bislang nur teilweise realisiert (Direktpfad-Kandidat plus Relay-Rückfall). Beide Punkte gehören zusammen: Erst mit mehreren Edges und einer belastbaren Mesh-Koordination wird die Providerblindheit auch gegen den Ausfall oder die Kompromittierung eines einzelnen Standorts robust.

Abrechnungsseitige Sybil-Resistenz. Proof-of-Work deckt das Fluten des Rendezvous ab, aber keinen finanziell ausgestatteten Angreifer: Wer den Rechenaufwand bezahlt, umgeht die Schranke. Solange auf KYC verzichtet wird, bleibt die abrechnungsseitige Missbrauchs- und Sybil-Resistenz die entscheidende offene Frage für einen tragfähigen Betrieb. Denkbar sind an das Guthabenbuch der Steuerungsebene gekoppelte Kosten- oder Reputationssignale, die Missbrauch verteuern, ohne die Pseudonymität am Datenpfad aufzugeben.

Breitere Evaluation. Die Messreihe deckt Verzögerung und Verlust in einem symmetrischen Einmal-Tunnel ab. Aussagekräftiger wären größere Stichproben, ein Keep-Alive-Datenpfad neben dem Frisch-Verbindungsaufbau, Bandbreitenbegrenzung sowie asymmetrische und bidirektionale Impairment. Auch

eine gezielte Vermessung des Failover-Fensters und der Rotationsdauer unter Last würde die Betriebsaussagen quantitativ untermauern.

8.3 Grenzen der Arbeit

Die Ergebnisse sind an mehreren Stellen bewusst begrenzt. *Erstens* ist die Providerblindheit strukturell, nicht metadatenfrei: Der Edge kann Nutzlast weder lesen noch verändern und erlangt keine Origin-Schlüssel, sieht aber die Existenz eines Tunnels sowie grobe Zeit- und Volumenzähler; ein Beobachter mit Zugriff auf diese Zähler kann daraus statistische Rückschlüsse ziehen. *Zweitens* ist die Schlüsselverteilung Sache des Kunden: Fähigkeiten werden außerhalb des Kanals ausgetauscht (ADR-0014), und die Rotation ist explizit – es gibt keine automatische Ablaufzeit, sondern ein bewusst getriggertes Fenster, in dem der zurückgezogene Schlüssel noch bedient wird. *Drittens* ist die Evaluation im Docker-Testbett mit `tc netem` emuliert und auf einer einzelnen Maschine ausgeführt; sie modelliert reale Wide-Area-Pfade nur näherungsweise und lässt Konkurrenzverkehr, reale Routerpuffer und Mehrbenutzerlast außen vor. *Viertens* bleiben die oben genannten offenen Punkte – finanzierter Angreifer, Einregion-Betrieb, teilweise realisierte Mesh-Ebene – ausdrücklich außerhalb der Belege dieser Arbeit. Der Kern ist gleichwohl erreicht: ein belegbar providerblinder Tunnel, der als betreibbarer Dienst mit Hochverfügbarkeit, Beobachtbarkeit, Selbstbedienungs-PKI, ausfallfreier Schlüsselrotation und HTTPS bis zum Origin real nachgewiesen ist.

Literatur

- [1] Apple Inc. *iCloud Private Relay Overview*. https://www.apple.com/privacy/docs/iCloud_Private_Relay_Overview_Dec2021.PDF. 2021. (Besucht am 18.07.2026).
- [2] Adam Back. “Hashcash – A Denial of Service Counter-Measure”. In: Technical report. 2002.
- [3] Cloudflare, Inc. *Cloudflare Tunnel Documentation*. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>. 2024. (Besucht am 17.07.2026).
- [4] Roger Dingledine, Nick Mathewson und Paul Syverson. “Tor: The Second-Generation Onion Router”. In: *Proceedings of the 13th USENIX Security Symposium*. USENIX Association, 2004.
- [5] Jason A. Donenfeld. “WireGuard: Next Generation Kernel Network Tunnel”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2017. DOI: 10.14722/ndss.2017.23160.
- [6] John R. Douceur. “The Sybil Attack”. In: *Peer-to-Peer Systems (IPTPS 2002)*. Bd. 2429. Lecture Notes in Computer Science. Springer, 2002, S. 251–260. DOI: 10.1007/3-540-45748-8_24.
- [7] David Fifield u. a. “Blocking-resistant communication through domain fronting”. In: *Proceedings on Privacy Enhancing Technologies (PoPETs) 2015.2* (2015), S. 46–64. DOI: 10.1515/popets-2015-0009.
- [8] Arturo Filastò und Jacob Appelbaum. “OONI: Open Observatory of Network Interference”. In: *2nd USENIX Workshop on Free and Open Communications on the Internet (FOCI)*. USENIX Association, 2012.
- [9] Jana Iyengar und Martin Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. Internet Engineering Task Force (IETF), 2021. DOI: 10.17487/RFC9000.
- [10] ngrok, Inc. *ngrok Documentation — Secure Ingress Platform*. <https://ngrok.com/docs>. 2024. (Besucht am 17.07.2026).
- [11] Tommy Pauly u. a. *Proxying IP in HTTP*. RFC 9484. Internet Engineering Task Force (IETF), 2023. DOI: 10.17487/RFC9484.
- [12] Trevor Perrin. *The Noise Protocol Framework*. <https://noiseprotocol.org/noise.html>. Revision 34. 2018.
- [13] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Internet Engineering Task Force (IETF), 2018. DOI: 10.17487/RFC8446.

Literatur

- [14] David Schinazi. *Proxying UDP in HTTP*. RFC 9298. Internet Engineering Task Force (IETF), 2022. DOI: 10.17487/RFC9298.
- [15] Shadowsocks Project. *Shadowsocks: A Secure SOCKS5 Proxy*. <https://shadowsocks.org/>. 2020. (Besucht am 17.07.2026).
- [16] Tailscale Inc. *Tailscale Funnel Documentation*. <https://tailscale.com/kb/1223/funnel>. 2024. (Besucht am 17.07.2026).
- [17] Martin Thomson und Sean Turner. *Using TLS to Secure QUIC*. RFC 9001. Internet Engineering Task Force (IETF), 2021. DOI: 10.17487/RFC9001.
- [18] Martin Thomson und Christopher A. Wood. *Oblivious HTTP*. RFC 9458. Internet Engineering Task Force (IETF), 2024. DOI: 10.17487/RFC9458.
- [19] Eric Wustrow u. a. “Telex: Anticensorship in the Network Infrastructure”. In: *Proceedings of the 20th USENIX Security Symposium*. USENIX Association, 2011.
- [20] Yawning Angel. *obfs4 — The obfoursicator: A Look-Like-Nothing Plug-gable Transport*. <https://gitlab.com/yawning/obfs4>. 2019. (Besucht am 17.07.2026).